CAPÍTULO 1

PROGRAMACIÓN MULTIPROCESO

CONTENIDOS

Procesos. Estados de un proceso. Gestión de procesos en Linux.

Creación de procesos en Java.

Programación concurrente.

Programación paralela y distribuida. Ejecución de tareas en paralelo con PVM.

OBJETIVOS

Conocer las características de un proceso y su ejecución por el sistema operativo.

Conocer las características y diferencias de la programación concurrente, paralela y distribuida.

Crear procesos en Linux y utilizar clases Java para crear procesos.

Desarrollar programas que ejecuten tareas en paralelo.

RESUMEN DEL CAPÍTULO

En este capítulo estudiaremos los procesos. Aprenderemos a gestionar los procesos que se están ejecutando en el sistema operativo. Utilizaremos clases para lanzar procesos. Conoceremos las características de la programación concurrente, paralela y distribuida y aprenderemos a crear programas paralelos con PVM.

1.1. INTRODUCCIÓN

Todos los ordenadores actuales realizan varias tareas a la vez, por ejemplo ejecutar un programa de procesador de textos, leer información de un disco duro, imprimir un documento por la impresora, visualizar información en pantalla, etc. Cuando un programa se carga en la memoria para su ejecución se convierte en un proceso.

En un sistema operativo **multiproceso** o **multitarea** se puede ejecutar más de un proceso (programa) a la vez, dando la sensación al usuario de que cada proceso es el único que se está ejecutando. La única forma de ejecutar varios procesos simultáneamente es tener varias CPUs (ya sea en una máquina o en varias). En los sistemas operativos con una única CPU se va alternando la ejecución de los procesos, es decir, se quita un proceso de la CPU, se ejecuta otro y se vuelve a colocar el primero sin que se entere de nada; esta operación se realiza tan rápido que parece que cada proceso tiene dedicación exclusiva.

La **programación multiproceso** tiene en cuenta la posibilidad de que múltiples procesos puedan estar ejecutándose simultáneamente sobre el mismo código de programa. Es decir desde una misma aplicación podemos realizar varias tareas de forma simultánea, o lo que es lo mismo, podemos dividir un proceso en varios subprocesos. En este capítulo aprenderemos a ejecutar varios procesos simultáneamente.

1.2. PROCESOS Y SISTEMA OPERATIVO

Se puede definir un **proceso** como un programa en ejecución. Consiste básicamente en el código ejecutable del programa, los datos y la pila del programa, el contador de programa, el puntero de pila y otros registros, y toda la información necesaria para ejecutar el programa.

Todos los programas que se ejecutan en el ordenador se organizan como un conjunto de procesos. El sistema operativo decide parar la ejecución de un proceso, por ejemplo porque ha consumido su tiempo de CPU, y arrancar la de otro. Cuando se suspende temporalmente la ejecución de un proceso debe rearrancarse posteriormente en el mismo estado en que se encontraba cuando se paró, esto implica que toda la información referente al proceso debe almacenarse en alguna parte.

El **BCP** es una estructura de datos llamada *Bloque de Control de Proceso* donde se almacena información acerca de un proceso:

- Identificación del proceso. Cada proceso que se inicia es referenciado por un identificador único.
- Estado del proceso.
- Contador de programa.
- Registros de CPU.
- Información de planificación de CPU como la prioridad del proceso.
- Información de gestión de memoria.
- Información contable como la cantidad de tiempo de CPU y tiempo real consumido.
- Información de estado de E/S como la lista de dispositivos asignados, archivos abiertos, etc.

Mediante el comando **ps** (*process status*) de Linux podemos ver parte de la información asociada a cada proceso. El siguiente ejemplo muestra los procesos actualmente vivos en la máquina, se muestran 2 procesos ejecutándose, uno es el shell y el otro es la ejecución de la orden ps:

```
mj@ubuntu-mj:~$ ps
  PID TTY          TIME CMD
 1906 pts/0    00:00:00 bash
 2362 pts/0    00:00:00 ps
```

PID: identificador del proceso:

TTY: terminal asociado del que lee y al que escribe. Si no hay aparece interrogación.

TIME: tiempo de ejecución asociado, es la cantidad total de tiempo de CPU que el proceso ha utilizado desde que nació.

CMD: nombre del proceso.

La orden **ps –f** muestra más información:

```
mj@ubuntu-mj:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
mj        1906  1298  0 18:06 pts/0    00:00:00 bash
mj        2363  1906  0 18:33 pts/0    00:00:00 ps -f
```

UID: nombre de usuario.

PPID: PID del padre de cada proceso.

C: porcentaje de recursos de CPU utilizado por el proceso.

STIME: hora de inicio del proceso.

La orden **ps –AF** muestra todos los procesos activos con todos los detalles; como en los ejemplos anteriores se puede observar que la última línea que a aparece es la del comando que se está ejecutando:

```
mj@ubuntu-mj:~$ ps -AF
UID        PID  PPID  C    SZ   RSS PSR STIME TTY          TIME CMD
root         1     0  0   703  1524   0 18:05 ?        00:00:00 /sbin/init
root         2     0  0     0     0   0 18:05 ?        00:00:00 [kthreadd]
root         3     2  0     0     0   0 18:05 ?        00:00:00 [migration/0]
root         4     2  0     0     0   0 18:05 ?        00:00:00 [ksoftirqd/0]
root         5     2  0     0     0   0 18:05 ?        00:00:00 [watchdog/0]
root         6     2  0     0     0   0 18:05 ?        00:00:00 [events/0]
root         7     2  0     0     0   0 18:05 ?        00:00:00 [cpuset]
root         8     2  0     0     0   0 18:05 ?        00:00:00 [khelper]
root         9     2  0     0     0   0 18:05 ?        00:00:00 [async/mgr]
. . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . .
postfix   1782   968  0  1457  1348   0 18:06 ?        00:00:00 pickup -l -t fi
root      1786  1510  0   904  1080   0 18:06 ?        00:00:00 hald-addon-stor
nobody    1846  1324  0 13116  2776   0 18:06 ?        00:00:00 /opt/lampp/bin/
mj        1847     1  0 10495  9648   0 18:06 ?        00:00:02 /usr/lib/gnome-
nobody    1849  1324  0 13048  2744   0 18:06 ?        00:00:00 /opt/lampp/bin/
. . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . .
mj        2497  2493  0   405   440   0 18:34 ?        00:00:00 heart -pid 2493
mj        2557  2379  0  4263 10404   0 18:34 ?        00:00:01 /usr/bin/python
```

```
root      2698   324  0   664   424   0 18:39 ?        00:00:00 udevd --daemon
root      2699   324  0   664   448   0 18:39 ?        00:00:00 udevd --daemon
mj        2720  1906  0   679  1044   0 18:49 pts/0    00:00:00 ps -AF
```

C: porcentaje de CPU utilizado por el proceso.

SZ: tamaño virtual de la imagen del proceso.

RSS: tamaño de la parte residente en memoria en kilobytes.

PSR: procesador que el proceso tiene actualmente asignado.

En Ubuntu desde la opción de menú **Sistema -> Administración -> Monitor del sistema** podemos acceder a la interfaz gráfica que nos muestra información sobre los procesos que se están ejecutando, véase Figura 1.1.

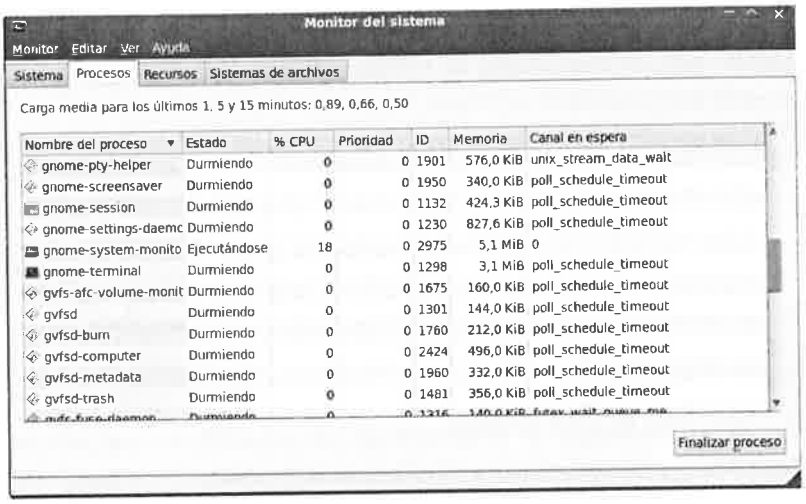


Figura 1.1. Monitor del sistema en Ubuntu.

En sistemas operativos Windows podemos usar desde la línea de comandos la orden **tasklist** para ver los procesos que se están ejecutando:

```
D:\>tasklist

Nombre de imagen              PID Nombre de sesión Núm. de  Uso de memor
========================= ====== ================ ======== ============
System Idle Process             0 Console                 0         28 KB
System                          4 Console                 0        340 KB
smss.exe                     1528 Console                 0        436 KB
csrss.exe                    1640 Console                 0      8.052 KB
winlogon.exe                 1664 Console                 0      1.736 KB
services.exe                 1708 Console                 0      3.676 KB
lsass.exe                    1720 Console                 0      1.412 KB
nvsvc32.exe                  1900 Console                 0      5.488 KB
svchost.exe                  1956 Console                 0      5.536 KB
svchost.exe                  2008 Console                 0      4.736 KB
svchost.exe                  1624 Console                 0     31.096 KB
svchost.exe                   248 Console                 0      4.036 KB
svchost.exe                   956 Console                 0      4.056 KB
spoolsv.exe                  1276 Console                 0      6.932 KB
explorer.exe                 1540 Console                 0     43.888 KB
RTHDCPL.exe                  1600 Console                 0     23.004 KB
rundll32.exe                 1636 Console                 0      4.364 KB
. . . . . .
```

La siguiente orden muestra los servicios que se están ejecutando bajo el proceso **svchost.exe**:

```
D:\>tasklist /svc /fi "imagename eq svchost.exe"

Nombre de imagen              PID Servicios
========================= ====== ============================================
svchost.exe                  1956 DcomLaunch, TermService
svchost.exe                  2008 RpcSs
svchost.exe                  1624 AudioSrv, Browser, CryptSvc, Dhcp, dmserver,
                                  EventSystem, FastUserSwitchingCompatibility,
                                  helpsvc, LanmanServer, lanmanworkstation,
                                  Netman, Nla, RasMan, Schedule, seclogon,
                                  SENS, SharedAccess, ShellHWDetection,
                                  TapiSrv, Themes, TrkWks, W32Time, winmgmt,
                                  wuauserv, WZCSVC
svchost.exe                   248 Dnscache
svchost.exe                   956 LmHosts, SSDPSRV
svchost.exe                  1608 WebClient
svchost.exe                  2872 stisvc
svchost.exe                  6108 HTTPFilter
```

Aunque lo más típico es usar la combinación de teclas [**CTRL+ Alt + Supr**] para que se muestre la pantalla que da acceso al *Administrador de tareas de Windows*, véase Figura 1.2.

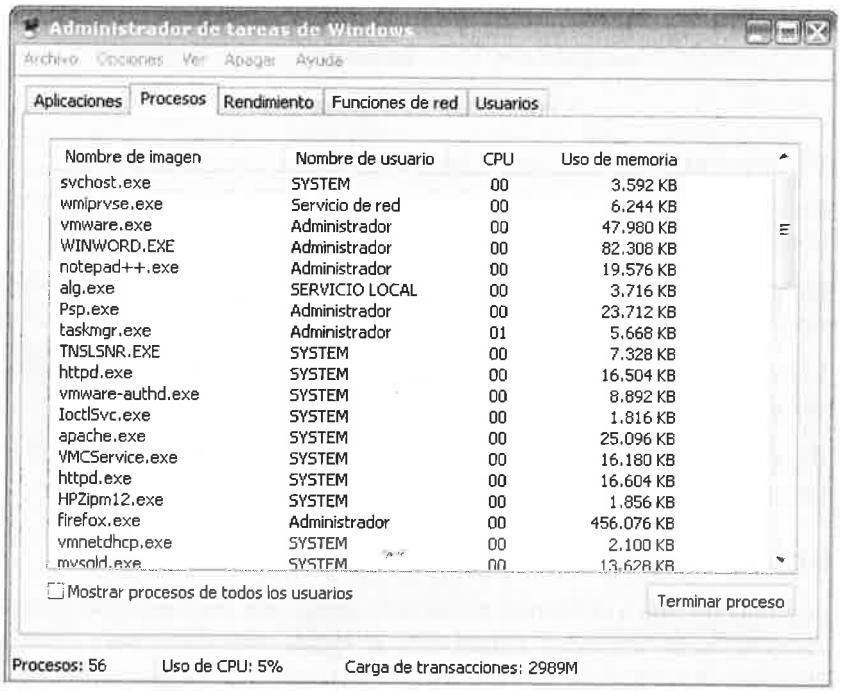


Figura 1.2. Administrador de tareas de Windows.

1.2.1. Estados de un proceso

Un proceso aunque es una entidad independiente puede generar una salida que se use como entrada para otro proceso. Entonces este segundo proceso tendrá que esperar a que el primero termine para obtener los datos a procesar, en este caso debe bloquearse hasta que sus datos de

entrada estén disponibles. Un proceso también se puede parar porque el sistema operativo decida asignar el procesador a otro proceso. En definitiva, los estados en los que se pueden encontrar un proceso son los siguientes:

- **En ejecución**: el proceso está actualmente ejecutándose, es decir, usando el procesador.
- **Bloqueado:** el proceso no puede hacer nada hasta que no ocurra un evento externo, como por ejemplo la finalización de una operación de E/S.
- **Listo:** el proceso está parado temporalmente y listo para ejecutarse cuando se le dé oportunidad.

La Figura 1.3 muestra mediante un diagrama de estados, los estados en que se puede encontrar un proceso.

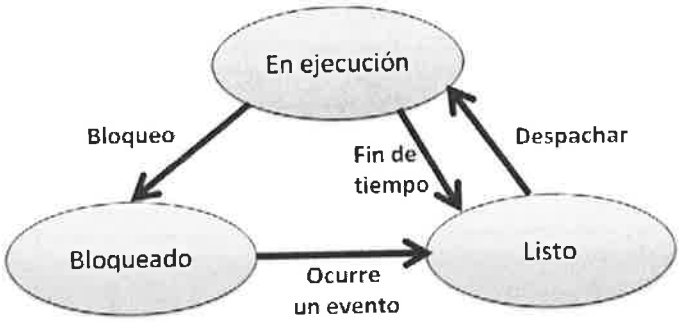


Figura 1.3. Estados de un proceso.

Las transiciones entre los estados son las siguientes:

- **En ejecución - Bloqueado**: un proceso pasa de ejecución a bloqueado cuando espera la ocurrencia de un evento externo.
- **Bloqueado - Listo:** un proceso pasa de bloqueado a listo cuando ocurre el evento externo que se esperaba.
- **Listo - En ejecución:** un proceso pasa de listo a ejecución cuando el sistema le otorga un tiempo de CPU.
- **En ejecución - Listo**: un proceso pasa de ejecución a listo cuando se le acaba el tiempo asignado por el sistema operativo.

1.2.2. Control de procesos en Linux

Seguro que más de una vez hemos necesitado dentro de un programa ejecutar otro programa que realice alguna tarea concreta. Linux ofrece varias funciones para realizar esto: *system()*, *fork()* y *execl()*.

La función ***system()*** se encuentra en la librería estándar *stdlib.h* por lo que funciona en cualquier sistema operativo que tenga un compilador de C/C++ como por ejemplo Linux, Windows, etc. El formato es el siguiente:

```
int system(const char *cadena)
```

La función recibe como parámetro una cadena de caracteres que indica el comando que se desea procesar. Dicha instrucción es pasada al intérprete de comandos del ambiente en el que se

esté trabajando y se ejecuta. Devuelve el valor -1 si ocurre un error y el estado devuelto por el comando en caso contrario.

La ejecución del siguiente ejemplo en C lista el contenido del directorio actual y lo envía a un fichero, abre el editor *gedit* con el fichero generado y ejecuta un comando que no existe en el intérprete de comandos de Linux:

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
  printf("Ejemplo de uso de system():");
  printf("\n\tListado del directorio actual y envio a un fichero:");
  printf("%d",system("ls > ficsalida"));
  printf("\n\tAbrimos con el gedit el fichero...");
  printf("%d",system("gedit ficsalida"));
  printf("\n\tEste comando es erróneo: %d",system("ged"));
  printf("\nFin de programa....\n");
}
```

Lo compilamos y lo ejecutamos desde Linux:

```
mj@ubuntu-mj:~$ gcc ejemploSystem.c -o ejemploSystem
mj@ubuntu-mj:~$ ./ejemploSystem
Ejemplo de uso de system():
      Listado del directorio actual y envio a un fichero:0
sh: ged: not found
      Abrimos con el gedit el fichero...0
      Este comando es erróneo: 32512
Fin de programa....
mj@ubuntu-mj:~$
```

Esta función no se debe usar desde un programa con privilegios de administrador porque pudiera ser que se emplearan valores extraños para algunas variables de entorno y podrían comprometer la integridad del sistema. En este caso se utiliza *execl()*.

La función ***execl()*** tiene otras 5 funciones relacionadas (que no se tratarán en el tema). Realiza la ejecución y terminación del comando. Su formato es:

```
#include <unistd.h>
int execl(const char *fichero, const char *arg0, ...,
            char *argn, (char *)NULL);
```

La función recibe el nombre del fichero que se va a ejecutar y luego los argumentos terminando con un puntero nulo. Devuelve -1 si ocurre algún error y en la variable global *errno* se pondrá el código de error adecuado. Por ejemplo, para ejecutar el comando */bin/ls -l* escribimos:

```
execl("/bin/ls", "ls", "-l", (char *)NULL);
```

A continuación se muestra un ejemplo de uso de la función:

```
#include <unistd.h>
#include <stdio.h>
void main()
```

```
{
        printf("Ejemplo de uso de exec():");
        printf("Los archivos en el directorio son:\n");
        execl("/bin/ls", "ls", "-l",(char *)NULL);
        printf("¡¡¡ Esto no se ejecuta !!!\n");
}
```

Compilamos y ejecutamos:

```
mj@ubuntu-mj:~$ gcc ejemploExec.c -o  ejemploExec
mj@ubuntu-mj:~$ ./ejemploExec
Ejemplo de uso de exec():Los archivos en el directorio son:
total 176
-rwxr-xr-x  1 mj   mj     7259 2012-11-15 17:17 a.out
-rw-r--r--  1 root root 59105 2011-10-11 16:07 core.img
drwxr-xr-x  2 mj   mj     4096 2012-11-14 22:24 Descargas
drwxr-xr-x  2 mj   mj     4096 2012-09-19 20:29 Documentos
drwxrwsr-x  9 mj   mj     4096 2012-11-15 00:36 eclipse
-rw-r--r--  1 mj   mj      919 2012-11-15 01:39 ejemplo.c
-rwxr-xr-x  1 mj   mj     7256 2012-11-15 18:44 ejemploExec
-rwxr-xr-x  1 mj   mj     1110 2012-11-15 18:43 ejemploExec.c
-rwxr-xr-x  1 mj   mj     7259 2012-11-15 17:45 ejemploSystem
. . . . . .
```

Para crear nuevos procesos se dispone de la función ***fork()*** sin ningún tipo de parámetros y que se trata en el siguiente apartado.

1.2.2.1. Creación y ejecución de procesos

Hasta ahora hemos visto funciones que ejecutaban comandos ya sea del intérprete de comandos o de ficheros en disco, a continuación veremos una función cuya misión es crear un proceso. Se trata de la función ***fork()***. Su sintaxis es:

```
#include <unistd.h>

pid_t fork(void);
```

Al llamar a esta función se crea un nuevo proceso (proceso hijo) que es una copia exacta en código y datos del proceso que ha realizado la llamada (el proceso padre), salvo el PID y la memoria que ocupa. Las variables del proceso hijo son una copia de las del padre, por lo que modificar una variable en uno de los procesos no se refleja en el otro (ya que tienen distintas memorias).

El valor devuelto por *fork()* es un valor numérico:

- Devuelve -1 si se produce algún error en la ejecución.
- Devuelve 0 si no se produce ningún error y nos encontramos en el proceso hijo.
- Devuelve el PID asignado al proceso hijo si no se produce ningún error y nos encontramos en el proceso padre.

Antes de hacer un ejemplo con la función *fork()* vamos a ver cómo obtener el identificador de un proceso o PID. Para ello usamos 2 funciones que devuelven un tipo *pid_t*. Las funciones son las siguientes:

```
pid_t getpid(void);
```

Devuelve el identificador del proceso que realiza la llamada, es decir, del proceso actual.

```
pid_t getppid(void);
```

Devuelve el identificador del proceso padre del proceso actual.

Veamos un simple ejemplo para ver los PID del proceso actual y del proceso padre:

```
#include <stdio.h>
#include <unistd.h>

void main(void)
{
   pid_t id_pactual, id_padre;

   id_pactual = getpid();
   id_padre = getppid();

   printf("PID de este proceso: %d\n", id_pactual);
   printf("PID del proceso padre: %d\n", id_padre);
}
```

Lo compilamos y ejecutamos. Se visualiza una salida similar a:

```
mj@ubuntu-mj:~$ gcc ejemploPadres.c -o ejemploPadres
mj@ubuntu-mj:~$ ./ejemploPadres
PID de este proceso: 2833
PID del proceso padre: 1923
```

Si ejecutamos el comando **ps** para ver los procesos que se están ejecutando, podemos ver que el PID del shell de Ubuntu (1923) coincide con el padre del proceso ejecutado anteriormente:

```
mj@ubuntu-mj:~$ ps
  PID TTY          TIME CMD
 1923 pts/0    00:00:00 bash
 2834 pts/0    00:00:00 ps
```

A continuación vamos a ver un ejemplo donde el proceso actual (proceso padre) crea un proceso (proceso hijo) con la función *fork()* (Figura 1.4):

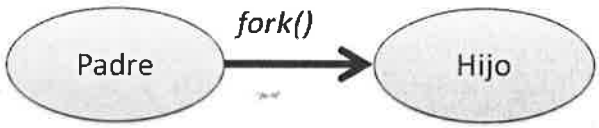


Figura 1.4. Proceso padre crea un hijo.

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
void main() {
  pid_t pid, Hijo_pid;
  pid = fork();

  if (pid == -1 ) //Ha ocurrido un error
  {
    printf("No se ha podido crear el proceso hijo...");
```

```
    exit(-1);
  }
  if (pid == 0 )  //Nos encontramos en Proceso hijo
  {
    printf("Soy el proceso hijo \n\t
            Mi PID es %d, El PID de mi padre es: %d.\n",
            getpid(), getppid() );
  }
  else    //Nos encontramos en Proceso padre
  {
   Hijo_pid = wait(NULL); //espera la finalización del proceso hijo
   printf("Soy el proceso padre:\n\t
           Mi PID es %d, El PID de mi padre es: %d.\n\t
           Mi hijo: %d terminó.\n",
           getpid(), getppid(), pid);
   }
   exit(0);
}
```

Lo compilamos y ejecutamos. Se visualiza una salida similar a:

```
mj@ubuntu-mj:~$ gcc ejemplo1Fork.c -o ejemplo1Fork
mj@ubuntu-mj:~$ ./ejemplo1Fork
Soy el proceso hijo
       Mi PID es 3098, El PID de mi padre es: 3097.
Soy el proceso padre:
       Mi PID es 3097, El PID de mi padre es: 1923.
       Mi hijo: 3098 terminó.
mj@ubuntu-mj:~$ ps
```

En el código anterior se utiliza la función ***wait()*** para que el proceso padre espere la finalización del proceso hijo, el proceso padre quedará bloqueado hasta que termine el hijo. La sintaxis de la orden es la siguiente:

```
pid_t wait(int *status);
```

Devuelve el identificador del proceso hijo cuya ejecución ha finalizado. La sentencia *wait(NULL)* es la forma más básica de esperar a que un hijo termine.

Partiendo del ejemplo anterior, creamos un nuevo proceso en el proceso hijo; así tendremos el proceso padre (ABUELO), el proceso hijo (HIJO) y el proceso hijo del hijo (NIETO) (Figura1.5):



Figura 1.5. Procesos Abuelo-Hijo-Nieto.

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
//ABUELO-HIJO-NIETO
void main() {
  pid_t pid, Hijo_pid, pid2, Hijo2_pid;

  pid = fork(); //Soy el  Abuelo, creo a Hijo

  if (pid == -1 ) //Ha ocurrido un error
  {
    printf("No se ha podido crear el proceso hijo...");
    exit(-1);
  }

  if (pid == 0 )  //Nos encontramos en Proceso hijo
  {
    pid2 = fork();//Soy el Hijo, creo a Nieto
    switch(pid2)
    {
      case -1:   // error
         printf("No se ha podido crear el proceso hijo
                 en el HIJO...");
         exit(-1);
         break;
      case 0:    // proceso hijo
         printf("\t\tSoy el proceso NIETO %d; Mi padre es = %d \n",
                     getpid(), getppid());
         break;
      default:   // proceso padre
        Hijo2_pid=wait(NULL);
        printf("\tSoy el proceso HIJO %d, Mi padre es: %d.\n",
                  getpid(), getppid());
        printf("\tMi hijo: %d terminó.\n", Hijo2_pid);
    }
  }

  else    //Nos encontramos en Proceso padre
  {
   Hijo_pid = wait(NULL); //espera la finalización del proceso hijo
   printf("Soy el proceso ABUELO: %d, Mi HIJO: %d terminó.\n",
           getpid(),  pid);
   }
   exit(0);
}
```

La compilación y ejecución muestra la siguiente salida:

```
mj@ubuntu-mj:~$ gcc ejemplo1_2Fork.c -o ejemplo1_2Fork
mj@ubuntu-mj:~$ ./ejemplo1_2Fork
            Soy el proceso NIETO 4486; Mi padre es = 4485
      Soy el proceso HIJO 4485, Mi padre es: 4484.
      Mi hijo: 4486 terminó.
Soy el proceso ABUELO: 4484, Mi HIJO: 4485 terminó.
mj@ubuntu-mj:~$
```

ACTIVIDAD 1.1

Realiza un programa en C que cree un proceso (tendremos 2 procesos uno padre y otro hijo). El programa definirá una variable entera y le dará el valor 6. El proceso padre incrementará dicho valor en 5 y el hijo restará 5. Se deben mostrar los valores en pantalla. A continuación se muestra un ejemplo de la ejecución:

```
mj@ubuntu-mj:~$ gcc actividad1_1.c -o actividad1_1
mj@ubuntu-mj:~$ ./actividad1_1
Valor inicial de la variable: 6
Variable en Proceso Hijo: 1
Variable en Proceso Padre: 11
mj@ubuntu-mj:~$
```

1.2.2.2. Comunicación entre procesos

Existen varias formas de comunicación entre procesos (*Inter-Process Communication* o IPC) de Linux: pipes, colas de mensajes, semáforos y segmentos de memoria compartida. En este tema trataremos los mecanismos más sencillos, los **pipes** (*tuberías* en castellano).

PIPES SIN NOMBRE

Un *pipe* es una especie de falso fichero que sirve para conectar dos procesos, véase Figura 1.6. Si el proceso A quiere enviar datos al proceso B, los escribe en el *pipe* como si este fuera un fichero de salida. El proceso B puede leer los datos sin más que leer el *pipe* como si se tratara de un fichero de entrada. Así la comunicación entre procesos es parecida a la lectura y escritura en ficheros normales.

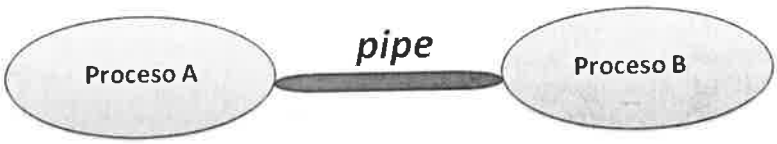


Figura 1.6. Dos procesos conectados por un *pipe*.

Cuando un proceso quiere leer del *pipe* y este está vacío, tendrá que esperar (es decir, se bloqueará) hasta que algún otro proceso ponga datos en él. Igualmente cuando un proceso intenta escribir en el *pipe* y está lleno se bloqueará hasta que se vacíe. El pipe es bidireccional pero, cada proceso lo utiliza en una única dirección, es este caso, el kernel gestiona la sincronización.

Para crear un *pipe* se realiza una llamada a la función ***pipe()***:

```
#include <unistd.h>
int pipe(int fd[2]);
```

Esta función recibe un solo argumento, que es un array de dos enteros: *fd[0]* contiene el descriptor para lectura y *fd[1]* el de escritura. Si la función tiene éxito devuelve 0 y el array contendrá dos nuevos descriptores de archivos para ser usados por la tubería. Si ocurre algún error devuelve -1.

Para enviar datos al *pipe*, se usa la función ***write()***, y para recuperar datos del *pipe*, se usa la función ***read()***. La sintaxis es la siguiente:

```
int read( int fd, void *buf, int count );
int write( int fd, void *buf, int count );
```

read() intenta leer *count* bytes del descriptor de fichero definido en *fd*, para guardarlos en el buffer *buf*. Devuelve el número de bytes leídos; si comparamos este valor con la variable *count* podemos saber si ha conseguido leer tantos bytes como se pedían.

write() es muy similar. A *buf* le damos el valor de lo que queramos escribir, definimos su tamaño en *count* y especificamos el fichero en el que escribiremos en *fd*. Veamos a continuación un sencillo ejemplo que usa ficheros, pero antes se exponen las funciones que abren y cierran ficheros. Usamos la función ***open()*** para abrirlo y ***close()*** para cerrarlo, la sintaxis es la siguiente:

```
int open( const char *fichero, int modo );

int close(int fd);
```

open() abre el fichero indicado en la cadena *fichero* según el modo de acceso indicado en el entero *modo* (0 para lectura, 1 para escritura, 2 para lectura y escritura, etc). Devuelve -1 si ocurre algún error. Para cerrar el fichero usamos *close()* indicando entre paréntesis el descriptor de fichero a cerrar.

Se parte de la existencia de un fichero vacío de nombre *texto.txt*, el programa abre el fichero para escritura, escribe un saludo y después cierra el fichero. Posteriormente vuelve a abrir el fichero en modo lectura y hace un recorrido leyendo los bytes de uno en uno. Al finalizar la lectura se cierra el fichero. El programa es el siguiente:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void main(void)
{
  char saludo[] = "Un saludo!!!\n";
  char buffer[10];
  int fd, bytesleidos;

  fd=open("texto.txt",1);//fichero se abre solo para escritura

  if( fd == -1 )
  {
   printf("ERROR AL ABRIR EL FICHERO...\n");
   exit(-1);
  }

  printf("Escribo el saludo...\n");
  write(fd,saludo, strlen(saludo));
  close(fd); //cierro el fichero

  fd=open("texto.txt",0);//el fichero se abre solo para lectura
  printf("Contenido del Fichero: \n");

  //leo bytes de uno en uno y lo guardo en buffer
  bytesleidos= read(fd, buffer, 1);
  while (bytesleidos!=0){
       printf("%s", buffer); //pinto el byte leido
       bytesleidos= read(fd, buffer, 1);//leo otro byte
  }
  close(fd);
```

```
}
```

La compilación y ejecución muestra la siguiente salida:

```
administrador@ubuntu1:~$ gcc ejemWriteRead.c -o ejemWriteRead
administrador@ubuntu1:~$ ./ejemWriteRead
Escribo el saludo...
Contenido del Fichero:
Un saludo!!!
administrador@ubuntu1:~$
```

Una vez que sabemos cómo leer y escribir en ficheros, veamos algunos ejemplos usando *pipes*. En el primer ejemplo se crea un proceso hijo con *fork()*. El proceso hijo envía al proceso padre mediante el uso de *pipes* el mensaje *"Hola papi"* en el descriptor para escritura *fd[1]*, el proceso padre mediante el descriptor *fd[0]* lee los datos enviados por el hijo:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main()
{
 int  fd[2];
 char buffer[30];
 pid_t pid;

 pipe(fd); //se crea el pipe

 pid=fork(); //se crea el proceso hijo

 switch(pid) {
   case -1 : //ERROR
           printf("NO SE HA PODIDO CREAR HIJO...");
           exit(-1);
           break;
   case 0 : //HIJO
           printf("El HIJO escribe en el pipe...\n");
           write(fd[1], "Hola papi", 10);
           break;
   default : //PADRE
           wait(NULL); //espera que finalice proceso hijo
           printf("El PADRE lee del pipe...\n");
           read(fd[0], buffer, 10);
           printf("\tMensaje leido: %s\n",buffer);
           break;
 }
}
```

La compilación y ejecución muestra la siguiente salida:

```
administrador@ubuntu1:~$ gcc ejemploPipe1.c -o ejemploPipe1
administrador@ubuntu1:~$ ./ejemploPipe1
El HIJO escribe en el pipe...
El PADRE lee del pipe...
      Mensaje leido: Hola papi
administrador@ubuntu1:~$
```

Primero se crea la tubería con *pipe()* y a continuación el proceso hijo. Recordemos que cuando se crea un proceso hijo con *fork()*, recibe una copia de todos los descriptores de ficheros del proceso padre, incluyendo copia de los descriptores de ficheros del pipe (*fd[0]* y *fd[1]*). Esto permite que el proceso hijo mande datos al extremo de escritura del pipe *fd[1]*, y el padre los reciba del extremo de lectura *fd[0]*, véase Figura1.7.

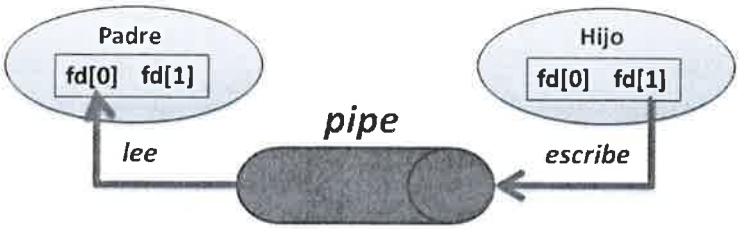


Figura 1.7. Pipe para un proceso hijo que escribe y otro padre que lee.

Los procesos padre e hijo están unidos por el pipe (Figura 1.8), pero la comunicación es en una única dirección, por tanto se debe decidir en qué dirección se envía la información, del padre al hijo o del hijo al padre; y dado que los descriptores se comparten siempre, debemos estar seguros de cerrar el extremo que no nos interesa.

Cuando el flujo de información va del padre hacia el hijo:

- El padre debe cerrar el descriptor de lectura *fd[0]*.
- El hijo debe cerrar el descriptor de escritura *fd[1]*.

Cuando el flujo de información va del hijo hacia padre ocurre lo contrario:

- El padre debe cerrar el descriptor de escritura *fd[1]*.
- El hijo debe cerrar el descriptor de lectura *fd[0]*.

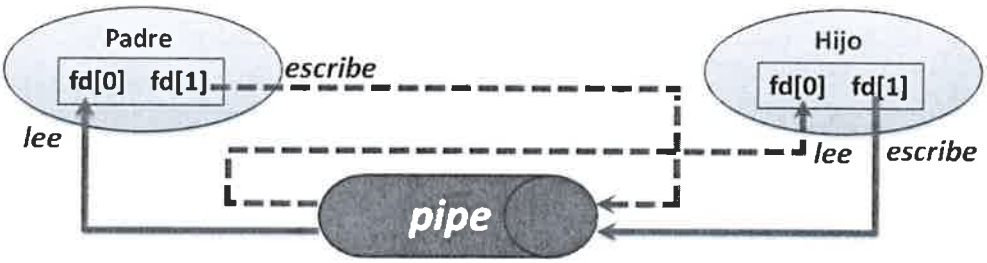


Figura 1.8. Pipe para un proceso padre e hijo que se envían datos.

El siguiente ejemplo crea un pipe en el que el padre envía un mensaje al hijo, el flujo de la información va del padre al hijo, el padre debe cerrar el descriptor *fd[0]* y el hijo *fd[1]*; el padre escribe en *fd[1]* y el hijo lee de *fd[0]*:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
```

```
int main(void)
{
  int fd[2];
  pid_t pid;
  char saludoPadre[]="Buenos dias hijo.\0";
  char buffer[80];

  pipe(fd);   //creo pipe
  pid=fork(); //creo proceso

  switch(pid) {
   case -1 : //ERROR
       printf("NO SE HA PODIDO CREAR HIJO...");
       exit(-1);

   case 0 : //HIJO RECIBE
       close(fd[1]);//cierra el descriptor de entrada
       read(fd[0], buffer, sizeof(buffer)); //leo el pipe
       printf("\tEl HIJO recibe algo del pipe: %s\n",buffer);
       break;

   default ://PADRE ENVIA
       close(fd[0]);
       write(fd[1],saludoPadre,strlen(saludoPadre));//escribo en pipe
       printf("El PADRE ENVIA MENSAJE AL HIJO...\n");
       wait(NULL); //espero al proceso hijo
       break;
 }
 return 0;
}
```

La compilación y ejecución muestra la siguiente salida:

```
administrador@ubuntu1:~$ gcc ejemploPipe3.c -o ejemploPipe3
administrador@ubuntu1:~$ ./ejemploPipe3
El PADRE ENVIA MENSAJE AL HIJO...
      El HIJO recibe algo del pipe: Buenos dias hijo.
administrador@ubuntu1:~$
```

ACTIVIDAD 1.2

Siguiendo el ejemplo anterior, realiza un programa en C que cree un pipe en el que el hijo envíe un mensaje al padre, es decir, la información fluya del hijo al padre. La ejecución debe mostrar la siguiente salida:

```
administrador@ubuntu1:~$ gcc actividad1_2.c -o actividad1_2
administrador@ubuntu1:~$ ./actividad1_2.c
      El HIJO envia algo al pipe.
El PADRE recibe algo del pipe: Buenos dias padre.
administrador@ubuntu1:~$
```

En el siguiente ejemplo vamos a hacer que padres e hijos puedan enviar y recibir información, como la comunicación es en un único sentido crearemos dos pipes *fd1* y *fd2*. Cada proceso usará un pipe para enviar la información y otro para recibirla. Partimos de los procesos ABUELO, HIJO y NIETO, la comunicación entre ellos se muestra en la Figura 1.9:



Figura 1.9. Pipe para un proceso padre e hijo que se envían datos.

- El ABUELO usará el *fd1* para enviar información al HIJO y recibirá la información de este a través del *fd2*.
- El HIJO usará el *fd2* para enviar información al NIETO y recibirá la información de este a través del *fd1*.
- El NIETO usará el *fd1* para enviar información al HIJO (su padre) y recibirá la información de este a través del *fd2*.

El código del programa es el siguiente:

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
//ABUELO-HIJO-NIETO
void main() {
  pid_t pid, Hijo_pid,pid2,Hijo2_pid;

  int fd1[2];
  int fd2[2];

  char saludoAbuelo[]="Saludos del Abuelo.\0";
  char saludoPadre[]= "Saludos del Padre..\0";
  char saludoHijo[]=  "Saludos del Hijo...\0";
  char saludoNieto[]= "Saludos del Nieto..\0";

  char buffer[80]="";

  pipe(fd1);
  pipe(fd2);
  pid = fork(); //Soy el  Abuelo, creo a Hijo

  if (pid == -1 ) //Ha ocurrido un error
  {
    printf("No se ha podido crear el proceso hijo...");
    exit(-1);
  }
```

```
if (pid == 0 )  //Nos encontramos en Proceso hijo
{
  pid2 = fork();//Soy el Hijo, creo a Nieto
  switch(pid2)
  {
    case -1:   // error
       printf("No se ha podido crear el proceso hijo en el HIJO.");
       exit(-1);
       break;
    case 0:  // proceso hijo (nieto)
       //NIETO RECIBE
       close(fd2[1]);//cierra el descriptor de entrada
       read(fd2[0], buffer, sizeof(buffer)); //leo el pipe
       printf("\t\tNIETO RECIBE mensaje de su padre:
                  %s\n",buffer);

        //NIETO ENVIA
        printf("\t\tNIETO ENVIA MENSAJE a su padre...\n");
        close(fd1[0]);
        write(fd1[1], saludoNieto, strlen(saludoNieto));
        break;
    default:   // proceso padre (hijo)
        //HIJO RECIBE
        close(fd1[1]);//cierra el descriptor de entrada
        read(fd1[0], buffer, sizeof(buffer)); //leo el pipe
        printf("\tHIJO recibe mensaje de ABUELO: %s\n",buffer);

        //HIJO ENVIA a su hijo
        close(fd2[0]);
        write(fd2[1], saludoPadre, strlen(saludoPadre));
        Hijo2_pid=wait(NULL); //espero al hijo

        //RECIBE de su hijo
        close(fd1[1]);//cierra el descriptor de entrada
        read(fd1[0], buffer, sizeof(buffer)); //leo el pipe
        printf("\tHIJO RECIBE mensaje de su hijo: %s\n",buffer);

        //HIJO ENVIA a su PADRE
        printf("\tHIJO ENVIA MENSAJE a su padre...\n",buffer);
        close(fd2[0]);
        write(fd2[1], saludoHijo, strlen(saludoHijo));
  }
}
else    //Nos encontramos en Proceso padre (abuelo)
{
 //PADRE ENVIA
 printf("ABUELO ENVIA MENSAJE AL HIJO...\n");
 close(fd1[0]);
 write(fd1[1], saludoAbuelo, strlen(saludoAbuelo));//escribo
 Hijo_pid = wait(NULL); //espera la finalización del hijo

 //PADRE RECIBE
 close(fd2[1]);//cierra el descriptor de entrada
 read(fd2[0], buffer, sizeof(buffer)); //leo el pipe
```

```
    printf("El ABUELO RECIBE MENSAJE del HIJO: %s\n", buffer);
  }
   exit(0);
}
```

La compilación y ejecución muestra la siguiente salida:

```
administrador@ubuntu1:~$ gcc ejemploForkPipe.c -o ejemploForkPipe
administrador@ubuntu1:~$ ./ejemploForkPipe
ABUELO ENVIA MENSAJE AL HIJO...
      HIJO recibe mensaje de ABUELO: Saludos del Abuelo.
            NIETO RECIBE mensaje de su padre: Saludos del Padre..
            NIETO ENVIA MENSAJE a su padre...
      HIJO RECIBE mensaje de su hijo: Saludos del Nieto..
      HIJO ENVIA MENSAJE a su padre...
El ABUELO RECIBE MENSAJE del HIJO: Saludos del Hijo...
administrador@ubuntu1:~$
```

PIPES CON NOMBRE O FIFOS (First In First Out)

Los pipes vistos anteriormente establecían un canal de comunicación entre procesos emparentados (padre-hijo). Los FIFOS permiten comunicar procesos que no tienen que estar emparentados.

Un FIFO es como un fichero con nombre que existe en el sistema de ficheros y que pueden abrir, leer y escribir múltiples procesos. Los datos escritos se leen como en una cola, primero en entrar (*FIRST IN*), primero en salir (*FIRST OUT*); y una vez leídos, no pueden ser leídos de nuevo. Los FIFOS tienen algunas diferencias con los ficheros:

- Una operación de escritura en un FIFO queda en espera hasta que el proceso pertinente abra el FIFO para iniciar la lectura.
- Solo se permite la escritura de información cuando un proceso vaya a recoger dicha información.

Hay varias formas de crear un FIFO: ejecutando el comando *mknod* desde la línea de comandos de Linux o desde un programa C usando la función *mknod()*.

Para usar *mknod* desde la línea de comandos de Linux seguimos el siguiente formato:

```
mknod [opciones] nombreFichero p
```

Donde:

nombreFichero es el nombre del FIFO. Las *opciones* pueden ser:

-m modo, --mode=modo: establece los permisos de los ficheros creados según el valor de *modo* (su comportamiento es similar al del comando *chmod*).

--help: muestra en la salida estándar ayuda sobre el modo de empleo del comando, y luego finaliza.

--version: muestra en la salida estándar información sobre la versión, y luego finaliza.

El siguiente ejemplo crea un FIFO llamado FIFO1 desde la línea de comandos y luego se muestra la información del fichero creado. Se puede observar el indicador "p" que aparece en la lista del directorio y el símbolo de pipe | detrás del nombre:

```
administrador@ubuntu1:~$ mknod FIFO1 p
```

```
administrador@ubuntu1:~$ l -l FIFO1
prw-r--r-- 1 administrador administrador 0 2012-11-16 17:09 FIFO1|
```

Desde el explorador de archivos el FIFO creado tiene el siguiente aspecto:



A continuación veamos cómo funciona el FIFO. Ejecuto desde la línea de comandos la orden *cat* con el nombre FIFO1:

```
administrador@ubuntu1:~$ cat FIFO1
```

Observamos que se queda a la espera. Abro una nueva terminal y ejecuto desde la línea de comandos la orden *l* para enviar la información del directorio al FIFO1:

```
administrador@ubuntu1:~$ l > FIFO1
```

Veremos que el *cat* que anteriormente estaba a la espera se ejecuta ya que ha recibido la información.

Para crear un FIFO en C, utilizamos la función *mknod()*. Su formato es el siguiente:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
int mknod(const char *pathname, mode_t modo, dev_t dev);
```

Donde:

pathname: es el nombre del dispositivo creado.

modo: especifica tanto los permisos de uso y el tipo de nodo que se creará. Debe ser una combinación (utilizando OR bit a bit) de uno de los tipos de fichero que se enumeran a continuación y los permisos para el nuevo nodo. El tipo de nodo debe ser uno de los siguientes:

- S_IFREG o 0: para especificar un fichero normal (que será creado vacío).
- S_IFCHR: para especificar un fichero especial de caracteres.
- S_IFBLK: un fichero especial de bloques.
- S_IFIFO: para crear un FIFO.

Si el tipo de fichero es S_IFCHR o S_IFBLK entonces *dev* debe especificar los números mayor y menor del fichero especial de dispositivo creado; en caso contrario, es ignorado. Si *pathname* ya existe, o es un enlace simbólico, esta llamada fallará devolviendo el error EEXIST.

La función *mknod()* devuelve 0 si ha funcionado correctamente, -1 si ha ocurrido un error.

A continuación se muestra un ejemplo de uso de FIFOS. El programa *fifocrea.c* crea un FIFO de nombre *FIFO2* y lee la información del FIFO; mientras no hay información quedará en espera. El programa *fifoescribe.c* escribe información en el FIFO. La Figura1.10 muestra la ejecución, primero se ejecuta *fifocrea* desde un terminal y después ejecutamos varias veces *fifoescribe* desde otro terminal.

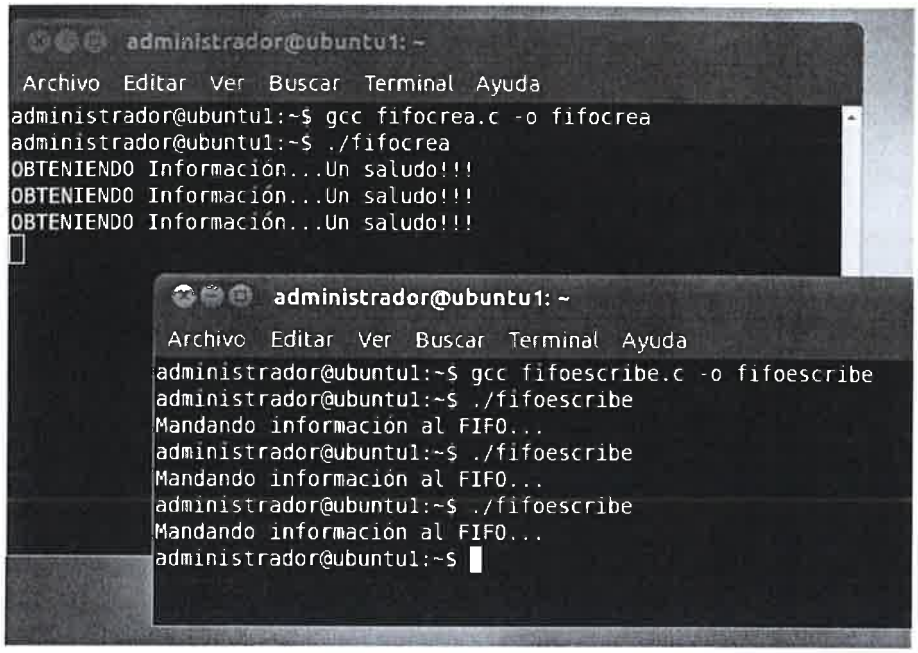


Figura 1.10. Ejecución de una FIFO.

El código es el siguiente:

```
//fifocrea.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int main(void)
{
  int fp;
  int p, bytesleidos;
  char saludo[] = "Un saludo!!!\n", buffer[10];

  p=mknod("FIFO2", S_IFIFO|0666, 0);//permiso de lectura y escritura

  if (p==-1) {
      printf("HA OCURRIDO UN ERROR...\n");
      exit(0);
  }

  while(1) {
   fp = open("FIFO2", 0);
   bytesleidos= read(fp, buffer, 1);
   printf("OBTENIENDO Información...");
   while (bytesleidos!=0){
       printf("%s", buffer);
       bytesleidos= read(fp, buffer, 1);//leo otro byte
   }
   close(fp);
  }
  return(0);
```

```
}

//fifoescribe.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
  int fp;
  char saludo[] = "Un saludo!!!\n";
  fp = open("FIFO2", 1);

  if(fp == -1) {
    printf("ERROR AL ABRIR EL FICHERO...");
    exit(1);
  }
  printf("Mandando información al FIFO...\n");
  write(fp,saludo, strlen(saludo));
  close(fp);
  return 0;
}
```

Con *mknod("FIFO2", S_IFIFO|0666, 0)* se crea un FIFO de nombre *FIFO2* con permisos de lectura y escritura.

1.2.2.3. Sincronización entre procesos

En el epígrafe anterior se han tratado los mecanismos más sencillos de comunicación entre procesos. Pero para que los procesos interactúen unos con otros necesitan cierto nivel de sincronización, es decir, es necesario que haya un funcionamiento coordinado entre los procesos a la hora de ejecutar alguna tarea. Podemos utilizar **señales** para llevar a cabo la sincronización entre dos procesos.

A continuación se muestran una serie de funciones útiles que utilizaremos para que un proceso padre y otro hijo se comuniquen de forma síncrona usando señales.

Una señal es como un aviso que un proceso manda a otro proceso. La función ***signal()*** es el gestor de señales por excelencia que especifica la acción que debe realizarse cuando un proceso recibe una señal. Su formato es el siguiente:

```
#include <signal.h>
void (*signal(int Señal, void (*Func)(int))(int);
```

Recibe dos parámetros:

Señal: contiene el número de señal que queremos capturar. En nuestro ejemplo pondremos SIGUSR1 que es una señal definida por el usuario para ser usada en programas de aplicación. Otra señal interesante es SIGKILL que se usa para terminar con un proceso.

Func: contiene la función a la que queremos que se llame. Esta función es conocida como el manejador de la señal (*signal handler*). En el ejemplo que se verá a continuación se definen dos manejadores de señal, uno para el proceso padre (*void gestion_padre(int segnal)*) y otro para el hijo (*void gestion_hijo(int segnal)*).

La función devuelve un puntero al manejador previamente instalado para esa señal. Un ejemplo de uso de la función: *signal(SIGUSR1, gestion_padre);* significa que cuando el proceso (en este caso el proceso padre) recibe una señal SIGUSR1 se realizará una llamada a la función *gestion_padre()*.

Para enviar una señal usaremos la función ***kill()***:

```
#include <signal.h>
int kill(int Pid, int Señal);
```

Recibe dos parámetros: el PID del proceso que recibirá la señal y la señal. Por ejemplo y suponiendo que *pid_padre* es el PID de un proceso padre: *kill(pid_padre, SIGUSR1);* envía una señal SIGUSR1 al proceso padre.

Cuando queremos que un proceso espere a que le llegue una señal, usamos la función ***pause()***. Para capturar esa señal, el proceso debe haber establecido un tratamiento de la misma con la función *signal()*. Este es su formato:

```
int pause(void);
```

Por último la función ***sleep()*** suspende al proceso que realiza la llamada la cantidad de segundos indicada o hasta que se reciba una señal.

```
#include <unistd.h>
 unsiged int sleep (unsigned int seconds);
```

En el siguiente ejemplo se crea un proceso hijo y el proceso padre le va a enviar dos señales SIGUSR1. Se define la función *manejador()* para gestionar la señal, visualizará un mensaje cuando el proceso hijo la reciba. En el proceso hijo se realiza la llamada a *signal()* donde se decide lo que se hará en el caso de recibir una señal, en este caso pinta un mensaje. Después hacemos un bucle infinito que no hace nada.

En el proceso padre se hacen las llamadas a *kill()* para enviar las señales. Con la función *sleep()* hacemos que los procesos esperen un segundo antes de continuar.

El código es el siguiente:

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <fcntl.h>
/*-----------------------------------------*/
/* gestión de señales en proceso HIJO      */
void manejador( int segnal )
{
 printf("Hijo recibe señal..%d\n", segnal);
}
/*-----------------------------------------*/
int main()
{
  int pid_hijo;
  pid_hijo = fork(); //creamos hijo

  switch(pid_hijo)
  {
```

```
    case -1:
         printf( "Error al crear el proceso hijo...\n");
         exit( -1 );
    case 0:   //HIJO
         signal( SIGUSR1, manejador); //MANEJADOR DE SEÑAL EN HIJO
         while(1) {
         };
    break;
    default: //PADRE envia 2 señales
         sleep(1);
         kill(pid_hijo, SIGUSR1);//ENVIA SEÑAL AL HIJO
         sleep(1);
         kill(pid_hijo, SIGUSR1);//ENVIA SEÑAL AL HIJO
         sleep(1);
    break;
  }
  return 0;
}
```

La compilación y ejecución muestra la siguiente salida:

```
administrador@ubuntu1:~$ gcc sincronizar-1.c -o sincronizar-1
administrador@ubuntu1:~$ ./sincronizar-1
Hijo recibe señal..10
Hijo recibe señal..10
administrador@ubuntu1:~$
```

En el ejemplo que se muestra a continuación un proceso padre y otro hijo se ejecutan de forma síncrona. Se han definido dos funciones para gestionar la señal uno para el padre y otro para el hijo, con las acciones que se realizarán cuando los procesos reciban una señal; en este caso se visualizará un mensaje.

En primer lugar el proceso padre crea el proceso hijo. Dentro de cada proceso se realiza una llamada a *signal()* donde se decide lo que se hará en el caso de recibir una señal. En el proceso padre tenemos las siguientes instrucciones donde se observa que entra en bucle infinito esperando a recibir una señal. Cuando recibe la señal se ejecutaría la función *gestion_padre()*. Con *kill()* envía la señal de respuesta al proceso hijo mediante su PID, y el proceso se vuelve a repetir:

```
signal( SIGUSR1, gestion_padre );
while(1)  {
    pause();//padre espera hasta recibir una señal del hijo
    sleep(1);
    kill(pid_hijo, SIGUSR1);//ENVIA SEÑAL AL HIJO
}
```

En el proceso hijo también tenemos un trozo de código parecido, por la colocación del *pause()* se puede deducir que es el proceso hijo el que inicia la comunicación con el padre mediante la llamada *kill()*. Primero envía la señal al padre y después espera a que le llegue una señal de respuesta, cuando recibe la señal ejecutaría la función *gestion_hijo()*:

```
signal( SIGUSR1, gestion_hijo );
while(1) { //bucle infinito
```

```
      sleep(1);
      kill(pid_padre, SIGUSR1);//ENVIA SEÑAL AL PADRE
      pause();//hijo espera hasta que llegue una señal de respuesta
  }
```

El código completo es el siguiente:

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <fcntl.h>
/*-----------------------------------------*/
/* gestión de señales en proceso padre      */
void gestion_padre( int segnal )
{
 printf("Padre recibe señal..%d\n", segnal);
}

/* gestión de señales en proceso hijo       */
void gestion_hijo( int segnal )
{
 printf("Hijo recibe señal..%d\n", segnal);
}
/*-----------------------------------------*/

int main()
{
  int pid_padre, pid_hijo;

  pid_padre = getpid();
  pid_hijo = fork(); //se crea el hijo

  switch(pid_hijo)
  {
    case -1:
     printf( "Error al crear el proceso hijo...\n");
     exit( -1 );

    case 0:   //HIJO
     //tratamiento de la señal en proceso hijo
     signal( SIGUSR1, gestion_hijo );
     while(1) { //bucle infinito
       sleep(1);
       kill(pid_padre, SIGUSR1);//ENVIA SEÑAL AL PADRE
       pause();//hijo espera hasta que llegue una señal de respuesta
     }
     break;

    default: //PADRE
     //tratamiento de la señal en proceso padre
     signal( SIGUSR1, gestion_padre );
     while(1)  {
        pause();//padre espera hasta recibir una señal del hijo
        sleep(1);
```

```
            kill(pid_hijo, SIGUSR1);//ENVIA SEÑAL AL HIJO
        }
        break;
    }
    return 0;
}
```

La Figura 1.11 muestra un momento de la ejecución de los procesos.



Figura 1.11. Ejecución de procesos sincronizados.

Para detener el proceso podemos pulsar las teclas [CTRL+C] o bien mediante el comando **ps** podemos ver el PID de los procesos padre e hijo que se están ejecutando:

```
administrador@ubuntu1:~$ ps -fe | grep sincronizar
1000      1678  1549  0 22:20 pts/0    00:00:00 ./sincronizar
1000      1679  1678  0 22:20 pts/0    00:00:00 ./sincronizar
1000      1687  1572  0 22:21 pts/1    00:00:00 grep --color=auto
                                                  sincronizar

administrador@ubuntu1:~$ kill 1679
administrador@ubuntu1:~$ kill 1678
administrador@ubuntu1:~$
```

Primero eliminaremos el proceso hijo (PID 1679) y después el padre (PID 1678). Al eliminar el hijo el padre quedará esperando.

ACTIVIDAD 1.3

Realiza un programa C en donde un hijo envíe 3 señales SIGUSR1 a su padre y después envíe una señal SIGKILL para que el proceso padre termine.

1.2.3. Creación de procesos con Java

Java dispone en el paquete *java.lang* de varias clases para la gestión de procesos: **Process** y **Runtime**. Cada aplicación Java que se ejecuta dispone de una instancia de la clase **Runtime** que representa el entorno de ejecución de la aplicación. Dos métodos muy importantes de esta clase son los siguientes:

- *static Runtime getRuntime()*: devuelve el objeto **Runtime** asociado con la aplicación Java en curso.
- *Process exec(String comando)*: ejecuta la orden especificada en *comando* en un proceso separado. Devuelve un objeto **Process**, que se puede utilizar para controlar la interacción del programa Java con el nuevo proceso en ejecución. La orden puede ser cualquier comando del sistema operativo. Puede lanzar varias excepciones: *SecurityException* (si existe un gestor de seguridad y no se permite la ejecución de subprocesos), *IOException* (error de E/S), *NullPointerException* (el comando es nulo) y *IllegalArgumentException* (el comando está vacío).

El siguiente ejemplo Java muestra cómo se puede ejecutar una aplicación de Windows, en este caso el NOTEPAD, que es el bloc de notas de Windows:

```
public class Ejemplo1 {
   public static void main(String[] args)  {
     Runtime r = Runtime.getRuntime();
     String comando= "NOTEPAD";
     Process p;
     try {
         p = r.exec( comando );
       } catch (Exception e) {
           System.out.println("Error en: "+comando );
           e.printStackTrace();
       }
   }
}//Ejemplo1
```

Lo compilamos y ejecutamos. Al ejecutarlo se abre el bloc de notas de Windows:

```
D:\CAPIT1>javac Ejemplo1.java
D:\CAPIT1>java Ejemplo1
```

Para los comandos de Windows que no tienen ejecutable (como por ejemplo DIR o ATTRIB) es necesario utilizar el comando CMD.EXE. Entonces para hacer un DIR desde un programa Java tendríamos que escribir en el comando lo siguiente:

```
String comando= "CMD /C DIR";
```

Sabías que...

CMD inicia una nueva instancia del intérprete de comandos de Windows. Para ver la sintaxis del comando escribimos desde el indicador del DOS: HELP CMD.

Para ejecutar un comando escribimos:

CMD /C comando: ejecuta el comando especificado y luego finaliza.

CMD /K comando: ejecuta el comando especificado pero sigue activo.

En el ejemplo anterior si queremos probar el comando CMD /C DIR no obtendremos ninguna salida. Es debido a que la salida del comando se redirige a nuestro programa Java, no a la

pantalla. Para leer la salida, es decir lo que nos devuelve el método *exec()* del **Runtime**, tenemos que usar el objeto **Process**, que se obtenía así en el ejemplo:

```
p = r.exec( comando );
```

La clase **Process** posee el método *getInputStream()* que nos permite leer el stream de salida del proceso, es decir, podemos leer lo que el comando que ejecutamos escribió en la consola. Definiremos así el stream:

```
Process p = Runtime.getRuntime().exec("CMD /C DIR");
InputStream is = p.getInputStream();
BufferedReader br = new BufferedReader (new InputStreamReader (is));
```

Para leer la salida usamos el método *readLine()* de **BufferedReader** que nos devuelve una línea de texto. A continuación se muestra la ejecución del comando DIR de Windows desde un programa Java:

```
import java.io.*;
public class Ejemplo2 {
      public static void main(String[] args)  {
        Runtime r = Runtime.getRuntime();
        String comando= "CMD /C DIR" ;
        Process p=null;
        try
        {
          p = r.exec( comando );
          InputStream is = p.getInputStream();
          BufferedReader br = new BufferedReader
                    (new InputStreamReader (is));
          String linea;
          while((linea = br.readLine())!=null) //lee una línea
                    System.out.println(linea);
          br.close();
        }
        catch (Exception e) {
              e.printStackTrace();
        }

        // COMPROBACION DE ERROR - 0 bien - 1 mal
        int exitVal;
        try {
            exitVal = p.waitFor();
            System.out.println("Valor de Salida: " + exitVal);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}//Ejemplo2
```

Lo compilamos y ejecutamos. Se visualiza lo siguiente:

```
D:\CAPIT1>javac Ejemplo2.java

D:\CAPIT1>java Ejemplo2
```

```
 El volumen de la unidad D es NUEVO
 El número de serie del volumen es: B87C-F34A

 Directorio de D:\CAPIT1

28/11/2012  21:51    <DIR>          .
28/11/2012  21:51    <DIR>          ..
28/11/2012  21:45               930 Ejemplo1.class
28/11/2012  21:45               309 Ejemplo1.java
28/11/2012  23:03             1.141 Ejemplo2.class
28/11/2012  23:03               670 Ejemplo2.java
               4 archivos          3.050 bytes
               2 dirs   4.032.409.600 bytes libres
Valor de Salida: 0
```

El método ***waitFor()*** hace que el proceso actual espere hasta que el subproceso representado por el objeto **Process** finalice. Devuelve 0 si ha finalizado correctamente.

ACTIVIDAD 1.4

Realiza un programa Java que ejecute el ejemplo (Ejemplo2) anterior usando **Runtime** y **Process**. Crea otro programa Java que reciba desde la línea de comandos un nombre y lo visualice. Utiliza el programa anterior para ejecutarlo.

La clase **Process** posee el método ***getErrorStream()*** que nos va a permitir obtener un stream para poder leer los posibles errores que se produzcan al lanzar el proceso. En el ejemplo anterior si cambiamos el comando y escribimos algo incorrecto, por ejemplo: *"CMD /C DIRR"* al ejecutarlo aparecerá como valor de salida 1 indicando que el proceso no ha finalizado correctamente:

```
D:\CAPIT1>java Ejemplo2
Valor de Salida: 1
```

Si añadimos el siguiente código al ejemplo:

```
try {
      InputStream er = p.getErrorStream();
      BufferedReader brer =
         new BufferedReader(new InputStreamReader(er));
      String liner = null;
      while ((liner = brer.readLine()) != null)
            System.out.println("ERROR >" + liner);
} catch (IOException ioe) {
            ioe.printStackTrace();
}
```

Se obtendrá la siguiente salida indicando el error que se ha producido:

```
D:\CAPIT1>java Ejemplo2
ERROR >"DIRR" no se reconoce como un comando interno o externo,
ERROR >programa o archivo por lotes ejecutable.
Valor de Salida: 1
```

Supongamos que tenemos un programa Java que pinta 5 veces un saludo que se envía desde la línea de comandos:

```
public class Unsaludo {
  public static void main(String[] args)  {
    if (args.length < 1) {
      System.out.println("SE NECESITA UN SALUDO...");
      System.exit(1);
    }
    for(int i=0; i<5; i++)
        System.out.println(i+1 +". "+args[0]);
  }
}//Unsaludo
```

Podemos ejecutar este programa y redireccionar la salida a un fichero:

```
D:\CAPIT1>Javac Unsaludo.java
D:\CAPIT1>Java Unsaludo "Hola Mundo!!" > fic.txt
```

Se habrá creado en el disco un fichero de nombre *fic.txt* con la salida generada por el programa. Si en el Ejemplo2 cambiamos el comando y escribimos lo siguiente: *String comando= "java Unsaludo \"Hola Mundo!!\" > ejem.txt";* al ejecutarlo veremos que no se genera el archivo de salida *ejem.txt*.

Y es que el método *exec()* no actúa como un intérprete de comandos o Shell, lo que hace es ejecutar un programa, nunca debemos utilizarlo como una línea de comandos. Si queremos que la salida del programa *Unsaludo* vaya a un fichero tendríamos que añadirlo mediante programación. El siguiente ejemplo toma la salida de la ejecución *Unsaludo* y la redirecciona hacia un fichero cuyo nombre se escribirá desde la línea de comandos. Utilizamos las clases **FileOutputStream** y **PrintWriter** para enviar la información al fichero:

```
import java.io.*;
public class Ejemplo3 {
 public static void main(String[] args)  {
  Runtime r = Runtime.getRuntime();
  String comando= "java Unsaludo \"Hola Mundo!!\" " ;
  Process p=null;

  if (args.length < 1) {
     System.out.println("SE NECESITA UN NOMBRE DE FICHERO...");
     System.exit(1);
   }

   try{
       //fichero al que se enviará la salida del programa Unsaludo
       FileOutputStream fos = new FileOutputStream(args[0]);
       PrintWriter pw = new PrintWriter(fos);
       p = r.exec( comando );

       InputStream is = p.getInputStream();
       BufferedReader br = new BufferedReader
                  (new InputStreamReader (is));
       String linea;
```

```
        while((linea = br.readLine())!=null) //lee la salida de Unsaludo
        {
          System.out.println("INSERTO EN "+args[0]+" > "+linea);
          pw.println(linea); //la inserta en el fichero
        }
        br.close();
        pw.close();
    }
    catch (Exception e)  { e.printStackTrace(); }

    // COMPROBACION DE ERROR - 0 bien 1 -mal
    int exitVal;
    try {
      exitVal = p.waitFor();
      System.out.println("Valor de Salida: " + exitVal);
    } catch (InterruptedException e) { e.printStackTrace(); }
  }
}//Ejemplo3
```

La compilación y ejecución muestra la siguiente salida:

```
D:\CAPIT1>Javac Ejemplo3.java

D:\CAPIT1>Java Ejemplo3 fic.txt
INSERTO EN fic.txt > 1. Hola Mundo!!
INSERTO EN fic.txt > 2. Hola Mundo!!
INSERTO EN fic.txt > 3. Hola Mundo!!
INSERTO EN fic.txt > 4. Hola Mundo!!
INSERTO EN fic.txt > 5. Hola Mundo!!
Valor de Salida: 0
```

ACTIVIDAD 1.5

Guarda en un fichero de nombre SVCHOST.TXT los servicios que se están ejecutando bajo el proceso **svchost.exe**.

Supongamos ahora que queremos ejecutar un comando que necesita información de entrada. Por ejemplo, si ejecutamos DATE desde la línea de comandos y pulsamos la tecla [Intro] nos pide escribir una nueva fecha:

```
D:\CAPIT1>DATE
La fecha actual es: 01/12/2012
Escriba la nueva fecha: (dd-mm-aa) 02-12-12
```

La clase **Process** posee el método ***getOutputStream()*** que nos permite escribir en el stream de entrada del proceso, así podemos enviar datos a un comando. El siguiente ejemplo ejecuta el comando DATE y le da los valores 02-12-12:

```
import java.io.*;
public class Ejemplo5 {
   public static void main(String[] args)  {
      Runtime r = Runtime.getRuntime();
      String comando="CMD /C DATE ";
      Process p=null;
```

```
    try{
       p = r.exec( comando );
       //escritura -- envia entrada a DATE
       OutputStream os = p.getOutputStream();
       os.write("02-12-12".getBytes());
       os.flush(); //vacía el buffer de salida

       //lectura -- obtiene la salida de DATE
       InputStream is = p.getInputStream();
       BufferedReader br = new BufferedReader
              (new InputStreamReader (is));
       String linea;
       while((linea = br.readLine())!=null)
            System.out.println(linea);
       br.close();

     } catch (Exception e)  { e.printStackTrace(); }

      // COMPROBACION DE ERROR - 0 bien 1 -mal
     int exitVal;
     try {
          exitVal = p.waitFor();
          System.out.println("Valor de Salida: " + exitVal);
          } catch (InterruptedException e) {
               e.printStackTrace();
          }
   }
}//Ejemplo5
```

La compilación y ejecución muestra la siguiente salida:

```
D:\CAPIT1>javac Ejemplo5.java
D:\CAPIT1>java Ejemplo5
La fecha actual es: 01/12/2012
Escriba la nueva fecha: (dd-mm-aa) 02-12-12
Valor de Salida: 0
```

Supongamos que tenemos un programa Java que lee una cadena desde la entrada estándar y la visualiza:

```
import java.io.*;
public class EjemploLectura{
 public static void main (String [] args)
 {
   InputStreamReader in = new InputStreamReader(System.in);
   BufferedReader br = new BufferedReader (in);
   String texto;
   try {
    System.out.println("Introduce una cadena....");
    texto= br.readLine();
    System.out.println("Cadena escrita: "+texto);
    in.close();
   }catch (Exception e) { e.printStackTrace();}
 }
```

```
}//EjemploLectura
```

Con el método *getOutputStream()* podemos enviar datos a la entrada estándar del programa *EjemploLectura*. Por ejemplo, si queremos enviar la cadena "Hola Manuel" cambiaríamos dos cosas en el *Ejemplo5*: el comando y la llamada al método *write()*:

```
String comando="java EjemploLectura";
os.write("Hola Manuel\n".getBytes());
```

Cada línea que mandemos a *EjemploLectura* debe terminar con "\n", igual que cuando escribimos desde el terminal la lectura termina cuando pulsamos la tecla [Intro]. Suponiendo que hemos guardado estos cambios en *Ejemplo6.java*, la compilación y ejecución muestra la siguiente salida:

```
D:\CAPIT1>java Ejemplo6
Introduce una cadena....
Cadena escrita: Hola Manuel
Valor de Salida: 0
```

ACTIVIDAD 1.6

Escribe un programa Java que lea 2 números desde la entrada estándar y visualice su suma. Controla que se escriben los argumentos necesarios y que sean numéricos. Haz otro programa Java para ejecutar el anterior.

La versión 1.5 de JDK añade una nueva forma de creación y ejecución de procesos del sistema operativo mediante la clase **ProcessBuilder**. Igual que **Process** y **Runtime** pertenece al paquete *java.lang*. Cada instancia **ProcessBuilder** gestiona una colección de atributos de proceso. El método *start()* crea una nueva instancia de **Process** con esos atributos y puede ser invocado varias veces desde la misma instancia para crear nuevos subprocesos con atributos idénticos o relacionados.

Cada process builder gestiona los siguientes atributos de un proceso:

- Un comando. Es una lista de cadenas que representa el programa que se invoca y sus argumentos si los hay.
- Un entorno (*environment*) con sus variables.
- Un directorio de trabajo. El valor por defecto es el directorio de trabajo del proceso en curso.
- Una fuente de entrada estándar. Por defecto, el subproceso lee la entrada de una tubería. El código Java puede acceder a esta tubería a través de la secuencia de salida devuelta por *Process.getOutputStream ()* (anteriormente vimos cómo se usaba este método). Sin embargo, la entrada estándar puede ser redirigida a otra fuente con *redirectInput()*. En este caso, *Process.getOutputStream()* devolverá una secuencia de salida nulo.
- Un destino para la salida estándar y la salida de error. Por defecto, el subproceso escribe en las tuberías de la salida y el error estándar. El código Java puede acceder a estas tuberías a través de los flujos de entrada devueltos por *Process.getInputStream()* y *Process.getErrorStream()* (vimos anteriormente algunos ejemplos). Sin embargo, la

salida estándar y el error estándar puede ser redirigido a otros destinos utilizando *redirectOutput()* y *redirectError()*. En este caso, *Process.getInputStream()* y/o *Process.getErrorStream()* devuelven una secuencia de entrada nula.

- Una propiedad *redirectErrorStream*. Inicialmente, esta propiedad es false, lo que significa que la salida estándar y salida de error de un subproceso se envían a dos corrientes separadas, que se pueden acceder a través de los métodos *Process.getInputStream()* y *Process.getErrorStream()*.

Para iniciar un nuevo proceso que utiliza el directorio de trabajo y el entorno del proceso en curso escribimos la siguiente orden:

```
Process p = new ProcessBuilder("Comando", "Argum1").start();
```

Para usar **ProcessBuilder** en los ejemplos anteriores no es necesario usar el método *exec()* de **Runtime**, en el Ejemplo2 construimos y ejecutamos la orden de la siguiente manera:

```
ProcessBuilder pb = new ProcessBuilder("CMD", "/C","DIR");
Process p = pb.start();
```

O bien:

```
ProcessBuilder pb = new ProcessBuilder("CMD", "/C","DIR").start();
```

En el Ejemplo3 hacemos lo siguiente:

```
ProcessBuilder pb = new ProcessBuilder("java",
                                       "Unsaludo", "\"Hola Mundo!!\"");
```

Y así sucesivamente en el resto de ejemplos. El siguiente ejemplo usa varios métodos de la clase **ProcessBuilder**: *environment()* que devuelve las variables de entorno del proceso; el método *commad()* sin parámetros, que devuelve los argumentos del proceso definido en *test*; y con parámetros donde se define un nuevo proceso y sus argumentos. Después se ejecutará este último proceso:

```
import java.io.*;
import java.util.*;

public class EjemploPB1 {
  public static void main(String args[]) {
    ProcessBuilder test = new ProcessBuilder();
    Map entorno = test.environment();
    System.out.println("Variables de entorno:");
    System.out.println(entorno);

    test = new ProcessBuilder("java", "Unsaludo", "\"Hola Mundo!!\"");

    //devuelve el nombre del proceso y sus argumentos
    List l = test.command();
    Iterator iter = l.iterator();
    System.out.println("Argumentos del comando:");
    while (iter.hasNext())
      System.out.println(iter.next());
```

```
    //ejecutamos el comando DIR
    test = test.command("CMD","/C" ,"DIR");
    try {
      Process p= test.start();
      InputStream is = p.getInputStream();
      BufferedReader br = new BufferedReader
                          (new InputStreamReader (is));
      String linea;
      while((linea = br.readLine())!=null) //lee la salida stándar
           System.out.println(linea);
      br.close();
    }
    catch (Exception e)  { e.printStackTrace(); }
}
}// EjemploPB1
```

La compilación y ejecución muestra la siguiente salida:

```
Variables de entorno:
{USERPROFILE=C:\Documents and Settings\Administrador,
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH,
=ExitCode=00000000, SystemDrive=C:, TEMP=C:\Windows\Temp,
ProgramFiles=C:\Archivos de programa, Path=;C:\Archivos de
programa\Java\jdk1.7.0_09\bin, HOMEDRIVE=C:, PROCESSOR_REVISION=0f0d,
CLIENTNAME=Console, =C:=C:\CAPIT1\PB, USERDOMAIN=27B3C536C6564E2,
ALLUSERSPROFILE=C:\Documents and Settings\All Users,
PROCESSOR_IDENTIFIER=x86 Family 6 Model 15 Stepping 13, GenuineIntel,
SESSIONNAME=Console, TMP=C:\Windows\Temp, PROCESSOR_ARCHITECTURE=x86,
CommonProgramFiles=C:\Archivos de programa\Archivos comunes,
LOGONSERVER=\\27B3C536C6564E2, OS=Windows_NT, FP_NO_HOST_CHECK=NO,
HOMEPATH=\Documents and Settings\Administrador, PROMPT=$P$G,
PROCESSOR_LEVEL=6, COMPUTERNAME=27B3C536C6564E2, SystemRoot=C:\WINDOWS,
windir=C:\WINDOWS, NUMBER_OF_PROCESSORS=1, USERNAME=Administrador,
ComSpec=C:\WINDOWS\system32\cmd.exe, APPDATA=C:\Documents and
Settings\Administrador\Datos de programa}
Argumentos del comando:
java
Unsaludo
"Hola Mundo!!"
 El volumen de la unidad C no tiene etiqueta.
 El n£mero de serie del volumen es: 7801-EDC2

 Directorio de C:\CAPIT1\PB

03/12/2012  17:45    <DIR>          .
03/12/2012  17:45    <DIR>          ..
03/12/2012  17:43             1.739 EjemploPB1.class
03/12/2012  17:43             1.122 EjemploPB1.java
03/12/2012  17:43             1.524 PP
03/12/2012  17:45             1.071 PP.TEXT
               4 archivos          5.456 bytes
               2 dirs   4.581.724.160 bytes libres
```

La versión 1.7 de JDK proporciona métodos que nos permiten redirigir la salida estándar y de error a otro fichero. Se trata de los métodos ***redirectOutput()*** y ***redirectError()***. El siguiente ejemplo ejecuta el comando "CMD /C DIR" y envía la salida al fichero *salida.txt*, si ocurre algún error se envía a *error.txt*:

```
import java.io.File;
import java.io.IOException;

public class EjemploPB2 {
  public static void main(String args[]) throws IOException {
    ProcessBuilder pb = new ProcessBuilder("CMD","/C" ,"DIR");

    File fOut = new File("C:/CAPIT1/PB/salida.txt");
    File fErr = new File("C:/CAPIT1/PB/error.txt");

    pb.redirectOutput(fOut);
    pb.redirectError(fErr);
    pb.start();
  }
}// EjemploPB2
```

También podemos ejecutar varios comandos del sistema operativos dentro de un fichero BAT. El siguiente ejemplo ejecuta los comandos MS-DOS que se encuentran en el fichero *fichero.bat*. Se utiliza el método ***redirectInput()*** para indicar que la entrada al proceso se encuentra en un fichero, es decir la entrada para el comando CMD será el *fichero.bat*. La salida del proceso se envía al fichero *salida.txt* y la salida de error al fichero *error.txt*:

```
import java.io.File;
import java.io.IOException;

public class EjemploPB3 {
  public static void main(String args[]) throws IOException {
    ProcessBuilder pb = new ProcessBuilder("CMD");

    File fBat = new File("C:/CAPIT1/PB/fichero.bat");
    File fOut = new File("c:/CAPIT1/PB/salida.txt");
    File fErr = new File("c:/CAPIT1/PB/error.txt");

    pb.redirectInput(fBat);
    pb.redirectOutput(fOut);
    pb.redirectError(fErr);
    pb.start();

  }
}// EjemploPB3
```

Suponiendo que los comandos MS-DOS del *fichero.bat* son estos:

```
MKDIR NUEVO
CD NUEVO
ECHO CREO FICHERO > Mifichero.txt
DIR
DIRR
ECHO FIN COMANDOS
```

Donde se crea una carpeta, nos dirigimos a dicha carpeta, se crea el fichero *Mifichero.txt*, se hace un DIR del directorio actual, el siguiente comando DIRR es erróneo y se visualiza FIN COMANDOS. Al ejecutarlo el contenido del fichero de salida *salida.txt* es:

```
C:\CAPIT1\PB>MKDIR NUEVO

C:\CAPIT1\PB>CD NUEVO

C:\CAPIT1\PB\NUEVO>ECHO CREO FICHERO > Mifichero.txt

C:\CAPIT1\PB\NUEVO>DIR
 El volumen de la unidad C no tiene etiqueta.
 El n£mero de serie del volumen es: 7801-EDC2

 Directorio de C:\CAPIT1\PB\NUEVO

03/12/2012  18:46    <DIR>          .
03/12/2012  18:46    <DIR>          ..
03/12/2012  18:46                15 Mifichero.txt
               1 archivos             15 bytes
               2 dirs   4.576.415.744 bytes libres

C:\CAPIT1\PB\NUEVO>DIRR

C:\CAPIT1\PB\NUEVO>ECHO FIN COMANDOS
FIN COMANDOS
```

Y el del fichero de error *error.txt*:

```
"DIRR" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.
```

> **NOTA:** Recuerda que estos ejemplos no funcionarán en versiones inferiores a la versión 1.7 de JDK.

1.3. PROGRAMACIÓN CONCURRENTE

El diccionario *WordReference.com* (http://www.wordreference.com/definicion/) nos muestra varias acepciones de la palabra concurrencia. Nos quedamos con la tercera: "*Acaecimiento o concurso de varios sucesos en un mismo tiempo*". Si sustituimos sucesos por procesos ya tenemos una aproximación de lo que es la concurrencia en informática: la existencia simultánea de varios procesos en ejecución.

1.3.1. Programa y proceso

Al principio del tema se definió un **proceso** como un programa en ejecución. Y ¿qué es un programa?, podemos definir **programa** como un conjunto de instrucciones que se aplican a un conjunto de datos de entrada para obtener una salida. Un proceso es algo activo que cuenta con una serie de recursos asociados, en cambio un programa es algo pasivo, para que pueda hacer algo hay que ejecutarlo.

Pero un programa al ponerse en ejecución puede dar lugar a más de un proceso, cada uno ejecutando una parte del programa. Por ejemplo el navegador web, por un lado está controlando las acciones del usuario con la interfaz, por otro hace las peticiones al servidor web. Entonces cada vez que se ejecuta este programa crea 2 procesos.

En la Figura 1.12 existe un programa almacenado en disco y 3 instancias del mismo ejecutándose, por ejemplo por 3 usuarios diferentes. Cada instancia del programa es un proceso, por tanto existen 3 procesos independientes ejecutándose al mismo tiempo sobre el sistema operativo, tenemos entonces 3 procesos concurrentes.

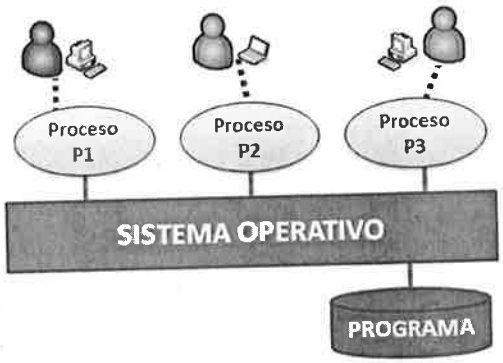


Figura 1.12. Un programa con 3 instancias ejecutándose.

Dos procesos serán concurrentes cuando la primera instrucción de uno de ellos se ejecuta después de la primera instrucción del otro y antes de la última. Es decir, existe un solapamiento o intercalado en la ejecución de sus instrucciones. No hay que confundir el solapamiento con la ejecución simultánea de las instrucciones, en este caso estaríamos en una situación de **programación paralela**, aunque a veces el hardware subyacente (más de un procesador) sí permitirá la ejecución simultánea.

Supongamos ahora que el programa anterior al ejecutarse da lugar a 2 procesos más, cada uno ejecutando una parte del programa, entonces la Figura 1.12 se convierte en la 1.13. Ya que un programa puede estar compuesto por diversos procesos, una definición más acertada de proceso es la de una actividad asíncrona susceptible de ser asignada a un procesador[1].

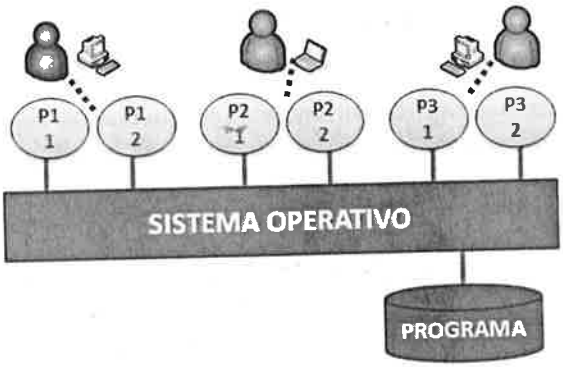


Figura 1.13. Un programa dando lugar a más de un proceso.

[1] Programación concurrente. José Tomás Palma Méndez y otros. Ed Paraninfo. ISBN: 9788497321846

Cuando varios procesos se ejecutan concurrentemente puede haber procesos que colaboren para un determinado fin (por ejemplo P1.1 y P1.2), y otros que compitan por los recursos del sistema (por ejemplo P2.1 y P3.1). Estas tareas de colaboración y competencia por los recursos exigen mecanismos de comunicación y sincronización entre procesos.

1.3.2. Características

La **programación concurrente** es la disciplina que se encarga del estudio de las notaciones que permiten especificar la ejecución concurrente de las acciones de un programa, así como las técnicas para resolver los problemas inherentes a la ejecución concurrente (comunicación y sincronización).

BENEFICIOS

La programación concurrente aporta una serie de beneficios:

Mejor aprovechamiento de la CPU. Un proceso puede aprovechar ciclos de CPU mientras otro realiza una operación de entrada/salida.

Velocidad de ejecución. Al subdividir un programa en procesos, estos se pueden "repartir" entre procesadores o gestionar en un único procesador según importancia.

Solución a problemas de naturaleza concurrente. Existen algunos problemas cuya solución es más fácil utilizando esta metodología:

- Sistemas de control: son sistemas en los que hay captura de datos, normalmente a través de sensores, análisis y actuación en función del análisis. Un ejemplo son los sistemas de tiempo real.
- Tecnologías web: los servidores web son capaces de atender múltiples peticiones de usuarios concurrentemente, también los servidores de chat, correo, los propios navegadores web, etc.
- Aplicaciones basabas en GUI: el usuario puede interactuar con la aplicación mientras la aplicación está realizando otra tarea. Por ejemplo el navegador web puede estar descargando un archivo mientras el usuario navega por las páginas.
- Simulación: programas que modelan sistemas físicos con autonomía.
- Sistemas Gestores de Bases de Datos: Los usuarios interactúan con el sistema, cada usuario puede ser visto como un proceso.

CONCURRENCIA Y HARDWARE

En un sistema **monoprocesador** (de un solo procesador) se puede tener una ejecución concurrente gestionando el tiempo de procesador para cada proceso. El S.O. va alternando el tiempo entre los distintos procesos, cuando uno necesita realizar una operación de entrada salida, lo abandona y otro lo ocupa; de esta forma se aprovechan los ciclos del procesador. En la Figura 1.14 se muestra cómo el tiempo de procesador es repartido entre 3 procesos, en cada momento solo hay un proceso. Esta forma de gestionar los procesos en un sistema monoprocesador recibe el nombre de **multiprogramación**.

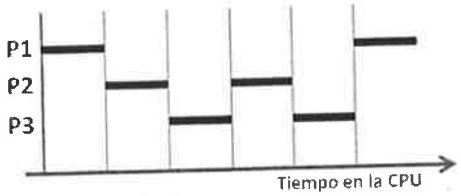


Figura 1.14. Concurrencia.

En un sistema **monoprocesador** todos los procesos comparten la misma memoria. La forma de comunicar y sincronizar procesos se realiza mediante variables compartidas.

En un sistemas **multiprocesador** (existe más de un procesador) podemos tener un proceso en cada procesador. Esto permite que exista paralelismo real entre los procesos, véase Figura 1.15. Estos pueden ser de memoria compartida (fuertemente acoplados) o con memoria local a cada procesador (débilmente acoplados). Se denomina **multiproceso** a la gestión de varios procesos dentro de un sistema multiprocesador, donde cada procesador puede acceder a una memoria común.

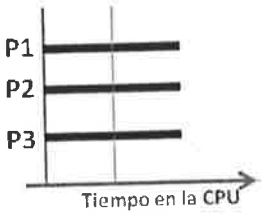


Figura 1.15. Paralelismo.

1.3.3. Programas concurrentes

Un **programa concurrente** define un conjunto de acciones que pueden ser ejecutadas simultáneamente. Supongamos que tenemos estas dos instrucciones en un programa, está claro que el orden de la ejecución de las mismas influirá en el resultado final:

x=x+1; y=x+1;	La primera instrucción se debe ejecutar antes de la segunda.

En cambio si tenemos estas otras, el orden de ejecución es indiferente:

x=1;
y=2;
z=3;
	El orden no interviene en el resultado final.

CONDICIONES DE BERNSTEIN

Bernstein definió unas condiciones para que dos conjuntos de instrucciones se puedan ejecutar concurrentemente. En primer lugar es necesario formar 2 conjuntos de instrucciones:

- **Conjunto de lectura:** formado por instrucciones que cuentan con variables a las que se accede en modo lectura durante su ejecución.
- **Conjunto de escritura:** formado por instrucciones que cuenta con variables a las que se accede en modo escritura durante su ejecución.

Por ejemplo, sean las siguientes instrucciones:

Instrucción 1:	x := y+1
Instrucción 2;	y := x+2
Instrucción 3:	z := a+b

Los conjuntos de lectura y escritura estarían formados por las variables siguientes:

	Conjunto lectura - L	Conjunto escritura - E
Instrucción 1- I1:	y	x
Instrucción 2- I2:	x	y
Instrucción 3- I3:	a,b	z

Se pueden expresar de la siguiente manera:

L(I1)={y}	E(I1)={x}
L(I2)={x}	E(I2)={y}
L(I3)={a,b}	E(I3)={z}

Para que dos conjuntos se puedan ejecutar concurrentemente se deben cumplir estas 3 condiciones:

- La intersección entre las variables leídas por un conjunto de instrucciones *Ii* y las variables escritas por otro conjunto *Ij* debe ser vacío, es decir, no debe haber variables comunes:

$$L(Ii) \cap E(Ij) = \emptyset$$

- La intersección entre las variables de escritura de un conjunto de instrucciones *Ii* y las variables leídas por otro conjunto *Ij* debe ser nulo, es decir, no debe haber variables comunes:

$$E(Ii) \cap L(Ij) = \emptyset$$

- Por último la intersección entre las variables de escritura de un conjunto de instrucciones *Ii* y las variables de escritura de un conjunto *Ij* debe ser vacío, no debe haber variables comunes:

$$E(Ii) \cap E(Ij) = \emptyset$$

En el ejemplo anterior tenemos las siguientes condiciones, donde se observa que las instrucciones I1 e I2 no se pueden ejecutar concurrentemente porque no cumplen las 3 condiciones:

Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) \neq \emptyset$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) \neq \emptyset$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$

En los programas secuenciales hay un orden fijo de ejecución de las instrucciones, siempre se sabe por dónde va a ir el programa. En cambio en los programas concurrentes hay un orden parcial. Al haber solapamiento de instrucciones no se sabe cuál va a ser el orden de ejecución, puede ocurrir que ante unos mismos datos de entrada el flujo de ejecución no sea el mismo. Esto

da lugar a que los programas concurrentes tengan un comportamiento indeterminista donde repetidas ejecuciones sobre un mismo conjunto de datos puedan dar diferentes resultados.

1.3.4. Problemas inherentes a la programación concurrente

A la hora de crear un programa concurrente podemos encontrarnos con dos problemas:

- **Exclusión mutua.** En programación concurrente es muy típico que varios procesos accedan a la vez a una variable compartida para actualizarla. Esto se debe evitar, ya que puede producir inconsistencia de datos: uno puede estar actualizando la variable a la vez que otro la puede estar leyendo. Por ello es necesario conseguir la exclusión mutua de los procesos respecto a la variable compartida. Para ello se propuso la **región crítica**. Cuando dos o más procesos comparten una variable, el acceso a dicha variable debe efectuarse siempre dentro de la región crítica asociada a la variable. Solo uno de los procesos podrá acceder para actualizarla y los demás deberán esperar, el tiempo de estancia es finito.
- **Condición de sincronización.** Hace referencia a la necesidad de coordinar los procesos con el fin de sincronizar sus actividades. Puede ocurrir que un proceso P1 llegue a un estado X que no pueda continuar su ejecución hasta que otro proceso P2 haya llegado a un estado Y de su ejecución. La programación concurrente proporciona mecanismos para bloquear procesos a la espera de que ocurra un evento y para desbloquearlos cuando este ocurra.

Algunas herramientas para manejar la concurrencia son: la región crítica, los semáforos, región crítica condicional, buzones, sucesos, monitores y sincronización por rendez-vous.

ACTIVIDAD 1.7

Responde a las siguientes cuestiones:

Escribe alguna característica de un programa concurrente.

¿Cuál es la ventaja de la concurrencia en los sistemas monoprocesador?

¿Cuáles son las diferencias entre multiprogramación y multiproceso?

¿Cuáles son los dos problemas principales inherentes a la programación concurrente?

1.3.5. Programación concurrente con Java

Al igual que el sistema operativo puede ejecutar varios procesos concurrentemente, dentro de un proceso podemos encontrarnos con varios hilos de ejecución. Un hilo es como una secuencia de control dentro de un proceso que ejecuta sus instrucciones de forma independiente, véase Figura1.16. Los hilos comparten el contexto del proceso, pero cada hilo mantiene una parte local.

Entre procesos e hilos hay algunas diferencias:

- Los hilos comparten el espacio de memoria del usuario, muchos comparten datos y espacios de direcciones; a diferencia de los procesos que generalmente poseen espacios de memoria independientes e interactúan a través de mecanismos de comunicación dados por el sistema.
- Hilos y procesos pueden encontrarse en diferentes estados, pero los cambios de estado en los procesos son más costosos.

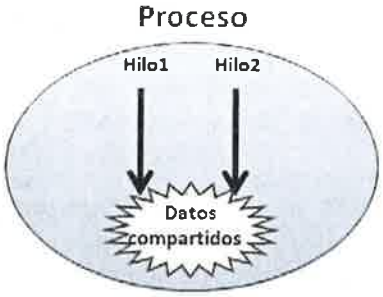


Figura 1.16. Hilos en un proceso.

Para programar concurrentemente podemos dividir nuestro programa en hilos. Java proporciona la construcción de programas concurrentes mediante la clase **Thread** (hilo o hebra). Esta clase permite ejecutar código en un hilo de ejecución independiente.

En Java existen dos formas de utilizar o crear un hilo:

- Creando una clase que herede de la clase **Thread** y sobrecargando el método *run()*.
- Implementando la interface **Runnable**, y declararando el método *run()*.

El siguiente ejemplo crea un hilo de nombre *HiloSimple* heredando de la clase **Thread**. En el método *run()* se indican las líneas de código que se ejecutarán simultáneamente con las otras partes del programa. Cuando se termina la ejecución de ese método, el hilo de ejecución termina también:

```
public class HiloSimple extends Thread {
   public void run() {
     for (int i=0; i<5;i++)
       System.out.println("En el Hilo... ");
   }
}
```

Para usar el hilo creo la clase *UsaHilo*:

```
public class UsaHilo {
    public static void main(String[] args) {
      HiloSimple hs = new HiloSimple();
      hs.start();
      for (int i=0; i<5;i++)
       System.out.println("Fuera del hilo..");
   }
}
```

Desde esta clase se arranca el hilo: primero se invoca al operador *new* para crear el hilo y luego al método *start()* que invoca al método *run()*. La compilación y ejecución muestra la siguiente salida, en la que se puede observar que se intercala las instrucciones del hilo y de fuera del hilo. La salida puede variar cada vez que vez que ejecutemos el programa:

```
D:\CAPIT1>javac HiloSimple.java
D:\CAPIT1>javac UsaHilo.java
D:\CAPIT1>java UsaHilo
Fuera del hilo..
Fuera del hilo..
```

```
Fuera del hilo..
En el Hilo...
En el Hilo...
En el Hilo...
En el Hilo...
En el Hilo...
Fuera del hilo..
Fuera del hilo..
```

Las 2 clases anteriores implementando la interfaz **Runnable** quedarían así:

```
public class HiloSimple2 implements Runnable{
   public void run() {
      for (int i=0; i<5;i++)
         System.out.println("En el Hilo...");
   }
}
public class UsaHilo2 {
    public static void main(String[] args) {
      HiloSimple2 hs = new HiloSimple2();
      Thread t = new Thread(hs);
      t.start();
      for (int i=0; i<5;i++)
        System.out.println("Fuera del hilo..");
   }
}
```

En el siguiente capítulo se tratarán más ampliamente los hilos con Java.

1.4. PROGRAMACIÓN PARALELA Y DISTRIBUIDA

1.4.1. Programación paralela

Un **programa paralelo** es un tipo de programa concurrente diseñado para ejecutarse en un sistema multiprocesador. El procesamiento paralelo permite que muchos elementos de proceso independientes trabajen simultáneamente para resolver un problema. Estos elementos pueden ser un número arbitrario de equipos conectados por una red, un único equipo con varios procesadores o una combinación de ambos. El problema a resolver se divide en partes independientes de tal forma que cada elemento pueda ejecutar la parte de programa que le corresponda a la vez que los demás.

Recordemos que en un sistema **multiprocesador,** donde existe más de un procesador, podemos tener un proceso en cada procesador y todos juntos trabajan para resolver un problema. Cada procesador realiza una parte del problema y necesita intercambiar información con el resto. Según cómo se realice este intercambio podemos tener modelos distintos de programación paralela:

- Modelo de **memoria compartida**: los procesadores comparten físicamente la memoria, es decir, todos acceden al mismo espacio de direcciones. Un valor escrito en memoria por un procesador puede ser leído directamente por cualquier otro.

- Modelo de **paso de mensajes**: cada procesador dispone de su propia memoria independiente del resto y accesible solo por él. Para realizar el intercambio de información es necesario que cada procesador realice la petición de datos al procesador que los tiene, y este haga el envío. El entorno de programación PVM que veremos más adelante utiliza este modelo.

Tradicionalmente, el paralelismo se ha utilizado en centros de supercomputación para resolver problemas de elevado coste computacional en un tiempo razonable, pero en la última década su interés se ha extendido por la difusión de los procesadores con múltiples núcleos.

> **NOTA:** El intercambio de información entre procesadores depende del sistema de almacenamiento que se disponga. Según este criterio las arquitecturas paralelas se clasifican en: *Sistemas de memoria compartida o multiprocesadores*: los procesadores comparten físicamente la memoria; y *Sistemas de memoria distribuida o multicomputadores*: cada procesador dispone de su propia memoria.

VENTAJAS E INCONVENIENTES

Ventajas del procesamiento paralelo:

- Proporciona ejecución simultánea de tareas.
- Disminuye el tiempo total de ejecución de una aplicación.
- Resolución de problemas complejos y de grandes dimensiones.
- Utilización de recursos no locales, por ejemplo los recursos que están en una red distribuida, una WAN o la propia red internet.
- Diminución de costos, en vez de gastar en un supercomputador muy caro se pueden utilizar otros recursos más baratos disponibles remotamente.

Pero no todo son ventajas, algunos inconvenientes son:

- Los compiladores y entornos de programación para sistemas paralelos son más difíciles de desarrollar.
- Los programas paralelos son más difíciles de escribir.
- El consumo de energía de los elementos que forman el sistema.
- Mayor complejidad en el acceso a los datos.
- La comunicación y la sincronización entre las diferentes subtareas.

La computación paralela resuelve problemas como: predicciones y estudios meteorológicos, estudio del genoma humano, modelado de la biosfera, predicciones sísmicas, simulación de moléculas... En algunos casos se dispone de tal cantidad de datos que sería muy lento o imposible tratar con máquinas convencionales.

ACTIVIDAD 1.8

Entra en la siguiente URL *https://computing.llnl.gov/tutorials/parallel_comp/* y responde a las siguientes cuestiones:

Cita algunas características de la computación serie.

Cita algunas características de la computación en paralelo.

Ámbitos en los que se usa la computación en paralelo.

¿Cómo hace uso de la computación paralela el proyecto SETI @ home?

1.4.2. Programación distribuida

Uno de los motivos principales para construir un sistema distribuido es compartir recursos. Probablemente, el sistema distribuido más conocido por todos es Internet que permite a los usuarios donde quiera que estén hacer uso de la World Wide Web, el correo electrónico y la transferencia de ficheros. Entre las aplicaciones más recientes de la computación distribuida se encuentra el *Cloud Computing* que es la computación en la nube o servicios en la nube, que ofrece servicios de computación a través de Internet.

Se define un sistema distribuido como aquel en el que los componentes hardware o software, localizados en computadores unidos mediante una red, comunican y coordinan sus acciones mediante el paso de mensajes. Esta definición tiene las siguientes consecuencias[2]:

- **Concurrencia:** lo normal en una red de ordenadores es la ejecución de programas concurrentes.
- **Inexistencia de reloj global:** cuando los programas necesitan cooperar coordinan sus acciones mediante el paso de mensajes.
- **Fallos independientes:** cada componente del sistema puede fallar independientemente, permitiendo que los demás continúen su ejecución.

La programación distribuida es un paradigma de programación enfocado en desarrollar sistemas distribuidos, abiertos, escalables, transparentes y tolerantes a fallos. Este paradigma es el resultado natural del uso de las computadoras y las redes. Casi cualquier lenguaje de programación que tenga acceso al máximo al hardware del sistema puede manejar la programación distribuida, considerando una buena cantidad de tiempo y código[3].

Una arquitectura típica para el desarrollo de sistemas distribuidos es la arquitectura **cliente-servidor**. Los clientes son elementos activos que demandan servicios a los servidores realizando peticiones y esperando la respuesta, los servidores son elementos pasivos que realizan las tareas bajo requerimientos de los clientes.

Por ejemplo, un cliente web solicita una página, el servidor web envía al cliente la página solicitada. Véase Figura 1.17. La comunicación entre servidores y clientes se realiza a través de la red.

[2] Sistemas Distribuidos: Conceptos y Diseño. George Coulouris y otros. Ed: Addison-Wesley.

[3] http://es.wikipedia.org/wiki/Programación_distribuida



Figura 1.17. Cliente-servidor sobre web.

Existen varios modelos de programación para la comunicación entre los procesos de un sistema distribuido:

- **Sockets.** Proporcionan los puntos extremos para la comunicación entre procesos. Es actualmente la base de la comunicación. Pero al ser de muy bajo nivel de abstracción, no son adecuados a nivel de aplicación. En el Capítulo 3 se tratarán los sockets en Java.
- **Llamada de procedimientos remotos o RPC** (*Remote Procedure Call*). Permite a un programa cliente llamar a un procedimiento de otro programa en ejecución en un proceso servidor. El proceso servidor define en su interfaz de servicio los procedimientos disponibles para ser llamados remotamente.
- **Invocación remota de objetos**. El modelo de programación basado en objetos ha sido extendido para permitir que los objetos de diferentes procesos se comuniquen uno con otro por medio de una *invocación a un método remoto* o **RMI** (*Remote Method Invocation*). Un objeto que vive en un proceso puede invocar métodos de un objeto que reside en otro proceso. **Java RMI** extiende el modelo de objetos de Java para proporcionar soporte de objetos distribuidos en lenguaje Java.

VENTAJAS E INCONVENIENTES

Ventajas que aportan los sistemas distribuidos:

- Se pueden compartir recursos y datos.
- Capacidad de crecimiento incremental.
- Mayor flexibilidad al poderse distribuir la carga de trabajo entre diferentes ordenadores.
- Alta disponibilidad.
- Soporte de aplicaciones inherentemente distribuidas.
- Carácter abierto y heterogéneo.

Pero no todo son ventajas, algunos inconvenientes son:

- Aumento de la complejidad, se necesita nuevo tipo de software.

- Problemas con las redes de comunicación: pérdida de mensajes, saturación del tráfico.
- Problemas de seguridad como por ejemplo, ataques de denegación de servicio en la que se "bombardea" un servicio con peticiones inútiles de forma que un usuario interesado en usar el servicio no pueda usarlo.

ACTIVIDAD 1.9
Busca en Internet aplicaciones de los sistemas distribuidos.

1.4.3. PVM. Instalación y configuración

PVM (*Paralel Virtual Machine* - Máquina virtual en paralelo) es un conjunto de herramientas software que permiten emular un marco de computación concurrente, distribuido y de propósito general, utilizando para ello grupos de ordenadores conectados, de manera que ni los ordenadores ni las redes que los conectan tienen las mismas características arquitectónicas.

Permite conectar entre sí ordenadores Unix y Windows (WIN95, NT 3.5, NT 4.0) para ser usados como un único gran ordenador paralelo de alto rendimiento. Así, grandes problemas de cómputo se pueden resolver de manera más rentable aprovechando la potencia y memoria de muchos equipos conectados.

Cientos de sitios en todo el mundo están usando PVM para resolver importantes problemas científicos, industriales y médicos, además de su uso como una herramienta educativa para enseñar programación paralela.

El modelo de computación de PVM se basa en considerar que una aplicación es una colección de tareas que se comunican y sincronizan mediante el paradigma de paso de mensajes. El sistema de PVM se compone de 3 partes[4]:

- *El demonio*, llamado **pvmd3** y a veces abreviado **PVMD**, que reside en todos los equipos que componen la máquina virtual. Uno de los equipos de la máquina virtual actúa como equipo anfitrión o maestro y los demás como esclavos, el maestro es el que inicia las tareas en paralelo. Figura 1.18.

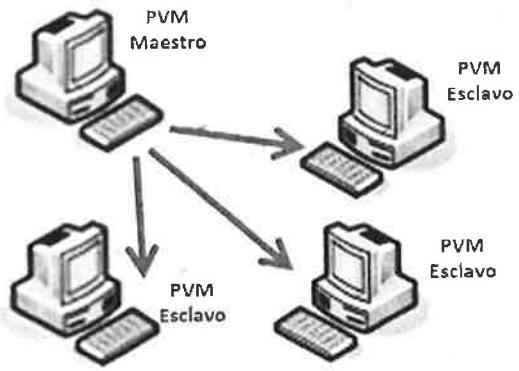


Figura 1.18. Máquina virtual PVM con varios equipos.

[4] Procesamiento paralelo teoría y programación. Sebastián Dormido Canto y otros, Ed: Sanz y Torres.